
Distilling Sequential Computation in Transformer Language Models

Zixuan Lan¹ Jessica Yang² Yanhong Li³ Karen Livescu² Jiawei Zhou⁴

¹The University of Chicago ²Toyota Technological Institute at Chicago

³Independent Researcher ⁴Stony Brook University

zixuanlan@uchicago.edu jiaminy@ttic.edu

yanhong.lbh@gmail.com klivescu@ttic.edu

jiawei.zhou.1@stonybrook.edu

Abstract

Transformer language models process sequences token by token in an autoregressive manner, making growing contexts increasingly expensive. Yet many adjacent token spans are highly predictable or frequently occur as stable units, suggesting that their representations may be compressible. We introduce a method for distilling sequential computation by replacing spans of input tokens with collapsed representations, computed on the fly by a lightweight merge module. This module generates a single surrogate embedding from a sequence of static token embeddings that captures the functional role of the multiple tokens, allowing pretrained models to operate on compressed inputs without architectural changes or re-training. We apply this approach during inference to compress both prompts and intermediate decoding steps, using a rollback mechanism to substitute stored multi-token KV cache entries with their single-step surrogates. Experiments across diverse models show that the merge module can be used to reduce effective sequence length by up to 40% with minimal accuracy degradation across language modeling evaluations and downstream tasks, including question answering, summarization, commonsense reasoning, and long-form mathematical reasoning. Additional lightweight adaptation of the merge module further improves the accuracy-compression trade-off in selected settings. These results demonstrate that sequential token computation in Transformers can be effectively approximated through condensed surrogate representations that preserve the original behavior without model updating.

1 Introduction

Autoregressive large language models (LLMs), typically built on Transformer architectures [Vaswani et al., 2017], process sequences token by token, making long inputs and growing autoregressive contexts expensive to compute and cache. Yet many adjacent token spans are highly predictable or frequently recurring in a sequence but are still processed as separate tokens. For example, “machine learning” or “New York” often appear together and are easy to predict once the first token is observed.

Prior work has explored reducing sequential computation in Transformer language models through alternative tokenization, multi-token decoding, and context reduction. Tokenization-based methods modify the input units on which models are trained, including byte- or patch-level representations [Pagnoni et al., 2024, Xue et al., 2022, Yu et al., 2023] and larger “superword” units [Liu et al., 2025], but typically require re-training because tokenization defines the model’s input structure. Other methods target pre-trained models through extended vocabularies, drafting models, or external chunk representations [Lan et al., 2023, Leviathan et al., 2023, Li et al., 2025b], but introduce auxiliary

components or inference overhead. Context-reduction methods prune or compress inputs, hidden states, or KV caches [Zhang et al., 2023, Bolya et al., 2023, Mu et al., 2023, Kallini et al., 2024, Jiang et al., 2023, Xiao et al., 2024, Shao et al., 2024], but often rely on model-internal signals, custom compression objectives, or accuracy-efficiency trade-offs. It remains unclear whether multi-step Transformer computation can be replaced at inference time with single-step surrogates without accessing internal activations or updating the pretrained backbone.

In this work, we address the above question by distilling the sequential computation of multiple tokens into a single-step input representation for pre-trained language models. Given a pre-trained language model (LM) and a span of input tokens in context during generation, we aim to construct a *single surrogate embedding* that replaces the original token span *at the input level*, while preserving the model’s next-token distribution. To handle the large number of possible token spans during inference, we introduce **Universal Multi-step Input Merging (UMIM)**, a shared, lightweight merge module that computes surrogate embeddings on demand. UMIM takes the static embeddings of the tokens in a selected span and computes a single surrogate embedding. The resulting surrogate is span-specific but context-independent: For the same n -grams, the module produces the same surrogate wherever it appears, while different n -grams naturally produce different surrogates.

We implement the UMIM module as a lightweight, single-layer attention network that serves as a plug-in to any pre-trained Transformer language model, applied to input embeddings *prior to model computation* at inference. UMIM is trained using a predictive distillation loss, encouraging the model’s output distribution with merged input embeddings to match that of the original uncompressed token sequences. Crucially, the pre-trained language model remains *frozen*, with only the external merge module updated during training. Once trained, the same UMIM module can be applied across different downstream tasks without re-training, offering a general and reusable mechanism for input-level sequential computation compression. Building on this general module, downstream performance can be further improved through task-specific merge-rule construction and lightweight adaptation of the merge module, while keeping the backbone language model frozen.

Using the shared UMIM module, we compress both static prompts and autoregressive decoding contexts, with rollback replacing multi-token KV-cache entries by single-step surrogate states. Experiments across diverse models and tasks show that UMIM directly reduces effective sequence length by up to 40% with minimal accuracy degradation, while preserving next-token predictive behavior.

2 Related Work

Tokenization Variations and Extension Prior work explores alternative tokenization strategies for Transformer language models. Byte-level and tokenizer-free models such as ByT5 [Xue et al., 2022], Megabyte [Yu et al., 2023], and ByteFusion [Pagnoni et al., 2024] reduce reliance on predefined vocabularies, while SuperBPE [Liu et al., 2025] introduces larger phrase-level units to reduce sequence length. Other methods extend token representations for span- or chunk-level decoding, including CoG [Lan et al., 2023], CD-LM [Li et al., 2025b], and adaptive hypertoken construction such as zip2zip [Geng et al., 2025]. However, these approaches generally require modifying the tokenizer, extending the token space, introducing auxiliary retrieval or encoding components, or retraining the model. In contrast, our method constructs surrogate embeddings for token spans post-training, without altering the tokenizer, vocabulary, or model architecture.

Latent Context Compression A closely related line of work compresses long contexts into learned continuous representations, including Gist Tokens [Mu et al., 2023], ICAE [Ge et al., 2024], Extensible Tokenization [Shao et al., 2024], 500xCompressor [Li et al., 2024], and multimodal processing of textual information via images [Li et al., 2025a]. These methods encode inputs into compact vectors or special-token representations that are later consumed by the language model. While effective, they typically require an additional encoding stage and treat compression as a separate interface. By contrast, our approach stays within the original token-based generation pipeline: It constructs surrogate embeddings for token spans directly at the input level and replaces multi-step computation with single-step surrogates during inference.

Prompt Compression and Context Reduction Prompt and context compression methods reduce inference-time computation through task-aware or task-agnostic content reduction. Task-aware methods such as LongLLMLingua [Jiang et al., 2024], Recomp [Xu et al., 2024], and also CPC

[Liskavets et al., 2024] retain content relevant to downstream questions or tasks. Task-agnostic methods, including ProCut [Xu et al., 2025], EHPC [Fei et al., 2025], LLMLingua [Jiang et al., 2023], LLMLingua2 [Pan et al., 2024], and self-information filtering [Li, 2023], estimate token or segment importance to remove less informative content. Other work studies abstractive prompt compression, where prompts are rewritten into shorter natural-language forms, as in Nano-Capsulator [Chuang et al., 2024] and Cmprsr [Zakazov et al., 2026]. In contrast, our approach is also task-agnostic but does not delete, filter, or rewrite input content.

KV Cache Management, Token Pruning and Merging Recent work reduces inference memory and computation through KV cache management, token pruning, or token merging. KV cache methods either statically filter tokens during prefill or dynamically retain, evict, or offload tokens during decoding, as in H₂O [Zhang et al., 2023], StreamingLLM [Xiao et al., 2024], KeyFormer [Adnan et al., 2024], OmniKV [Hao et al., 2025], PagedAttention [Kwon et al., 2023], and Blockwise Caching [Dao et al., 2022]. Other methods reduce context length through token pruning or merging, including MRT5 [Kallini et al., 2024] and hidden-state-based merging in language [Yuan et al., 2024], vision [Bolya et al., 2023, Shin et al., 2025], video [Ryoo et al., 2021]. In contrast, our method performs merging before Transformer computation rather than by pruning existing KV entries or combining hidden states during model execution. It replaces multi-token spans with single surrogate input embeddings and substitutes their corresponding multi-step KV cache entries with single-step surrogate states, preserving the functional role of the original tokens while reducing both effective sequence length and KV cache usage.

3 Methodology

We develop UMIM as an external plug-in for pre-trained Transformer language models, which maps selected contiguous token spans to single surrogate embeddings computed from their static input embeddings. Formally, let $\mathbf{x} = (x_1, \dots, x_T)$ be a sequence of input tokens, and let $e(\cdot)$ denote the token embedding lookup, applied element-wise so that $e(x_i)$ is a single embedding and $e(x_{a:b})$ denotes the corresponding sequence of embeddings. A Transformer language model with parameters θ processes the embedded prefix $e(x_{<t})$ and outputs the next-token distribution $p_\theta^t(\cdot \mid e(x_{<t}))$. We learn a merge module $\mathcal{M}_\phi$, parameterized by ϕ , that maps a span $x_{t:t+n}$ to a surrogate embedding $\tilde{e} = \mathcal{M}_\phi(e(x_{t:t+n}))$, which replaces the original embeddings while preserving the subsequent predictive distribution. The following sections describe merge span selection, the merge module architecture, the distillation objective, and decoding algorithms for applying UMIM during prompt compression and generation.

3.1 Merge Rule

To determine which token spans to merge, we construct a frequency-based merging set from a general corpus $\mathcal{C}$. Specifically, we collect unique contiguous n -grams of length $2 \leq n \leq 4$ whose frequency exceeds a threshold τ , forming the merging set $\mathcal{R}$. High-frequency spans are likely to recur during inference, enabling more frequent compression, and also provide sufficient training instances for learning surrogate embeddings. The maximum of $n = 4$ mitigates data sparsity while still allowing meaningful multi-token groupings. When multiple candidates overlap, we apply a longest-match strategy, giving precedence to the longest valid span at each position. During training and inference, spans that match entries in $\mathcal{R}$ are treated as merge candidates. The merge rule is independent of the model architecture and depends only on the tokenizer and corpus statistics. The corpus $\mathcal{C}$ used to construct $\mathcal{R}$ is task-independent, though in-domain data can further improve compression coverage by capturing domain-specific frequent spans. See Appendix B for details.

3.2 Lightweight Merge Module Architecture

The UMIM is a lightweight single-layer multi-head attention pooling network that operates solely on static input embeddings, prior to any Transformer computation. It serves as a universal replacement for contiguous token spans, compressing variable-length inputs into a single surrogate embedding via a *learnable query* mechanism.

Formally, let d denote the embedding dimension of the target language model, and h the number of attention heads. Each head has dimension $d_h = d/h$. Given the token span embeddings $e(x_{t:t+n}) \in$

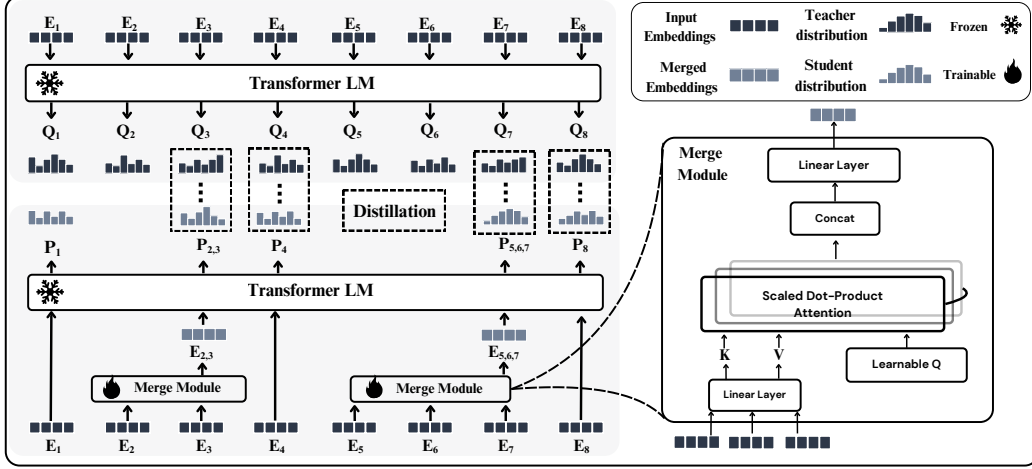


Figure 1: Training process of the UMIM module, with token-step aligned distillation loss and merge module architecture.

$\mathbb{R}^{d \times n}$, we use a learnable query parameter $Q \in \mathbb{R}^{h \times d_h}$, where the i^{th} row $q_i \in \mathbb{R}^{d_h}$ is the query vector for head i . We use a shared projection W^{kv} for span representations before attention pooling. After pooling, an output projection W^o maps the concatenated head outputs back to the model embedding space. The output surrogate embedding is computed as:

$$M_\phi(e(x_{t:t+n})) = W^o [o_1; \dots; o_h], \quad o_i = \text{softmax} \left(\frac{q_i W^{kv} e(x_{t:t+n})}{\sqrt{d_h}} \right) (W^{kv} e(x_{t:t+n}))^\top. \quad (1)$$

3.3 UMIM Learning by Distilling Sequential computation

We train the UMIM to produce single-step surrogate embeddings that can replace multi-tokens, while preserving the language model’s next-token distributions. The language model parameters θ are frozen throughout training. To guide learning, we introduce a *sequential computation distillation loss*, aligning predictions made on merged inputs with those from the original token-by-token computation. Let $\mathcal{M}_\phi(e(x_{1:T})) = (e'_s)_{s \in \mathcal{S}}$ denote the compressed sequence of token embeddings after applying the merge module to all eligible token spans according to the merge rule $\mathcal{R}$. Here, $\mathcal{S}$ represents the set of *retained token positions* after merging. For instance, if tokens $x_{t:t+n}$ are merged into a single surrogate embedding $\mathcal{M}_\phi(e(x_{t:t+n}))$, then token steps t through $t+n-1$ are collapsed, and only step $t+n-1$ is retained in $\mathcal{S}$ (e.g., $\mathcal{S} = (\dots, t-1, t+n-1, \dots)$).

To align the behavior of the compressed input with the original, we minimize the KL divergence between the output distributions of the language model on unmerged vs. merged input prefixes:

$$\mathcal{L}_{\text{distill}} = D_{\text{KL}}(p_\theta(\cdot | e(x_{\leq s})) \| p_\theta(\cdot | M_\phi(e(x_{\leq s})))) \quad (2)$$

This loss encourages the merged representation to produce predictive distributions that match those from the full token sequence. An illustration of the training procedure is shown in Figure 1.

3.4 Model Inference with Runtime Merging

Once trained, the UMIM module can be integrated into inference with the compatible pre-trained language model. During inference, UMIM reduces the effective sequence length by applying runtime merging in both prompt prefilling and autoregressive decoding as shown in Figure 2. Let $z = (z_1, \dots, z_m)$ denote the prompt tokens, and $y = (y_1, y_2, \dots)$ the generated tokens. During prefill, we scan the prompt for eligible n -grams in $\mathcal{R}$ and replace each matched span with its surrogate embedding. During decoding, after each new token is generated, we check whether the recent suffix forms an eligible n -gram; if so, we roll back the corresponding KV entries and replace them with a single merged KV state. The checking and rollback overhead is amortized over long sequences, while the reduced sequence length directly lowers KV-cache size and attention computation.

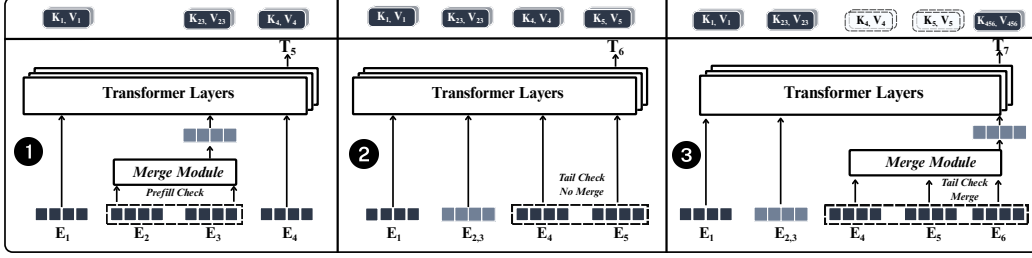


Figure 2: UIM Decoding Process: **E** is embedding, **T** is token, **K,V** is KV cache (1) During prefill, mergeable spans are replaced by surrogate embeddings. (2) During autoregressive decoding, each new token is checked against the recent suffix; if no valid merge is found, decoding proceeds normally by appending a new KV cache. (3) If the suffix forms a mergeable span, UIM rolls back the matched span and replaces its multiple KV entries with a single merged KV cache.

By default, we apply the merge rule underlying the training data $\mathcal{R}$ for inference. This, however, can be relaxed by adapting the merge rule further to a specific downstream task based on in-domain data, facilitating a wider range of compression ratio and domain generalization. See Section 4.5 for details.

4 Experiments

4.1 Experimental Setup

We train UIM modules for LMs including Llama 3.1 8B Grattafiori et al. [2024], Llama 3.2 1.5B, and GPT2-XL 1.5B Radford et al. [2019] using the WikiText-103 [Merity et al., 2016] corpus, and DeepScaleR-1.5B-Preview¹ using a long reasoning trajectory corpus we collected. Each merge module uses $h = 4$ attention heads and contains 7M–50M learnable parameters, depending on the backbone size. To build the merging rule $\mathcal{R}$, we extract $n \in \{2, 3, 4\}$ -grams with threshold $\tau = 5$, yielding over 2M unique token spans per model.

More details can be found in Appendix B

We first measure the distribution alignment quality after applying the embedding merges compared with the original token-based LLMs. The metrics include: Top-1 accuracy—percentage of token positions after merging where the top-1 tokens in the predictive distribution exactly match, Top-3/10 overlap—average number of overlapped tokens in the top-3/10 tokens from corresponding distributions at aligned token positions, Top-p overlap—the overlap between tokens whose cumulative probability mass reaches p in each distribution, and Mean Reciprocal Rank (MRR)—the average reciprocal rank of the teacher (original LLM)’s top-1 token within the distribution after span merging at corresponding token positions.

Table 1: Distribution alignment between the original LLMs and the merged-input models. We report all in percentage (%). See Sec. 4.1 for metric definitions.

Model	TR	Top-1	Top-3	Top-p	MRR
Llama-3.1-8B	37.8	79.2	77.6	92.9	87.1
Llama-3.2-1.5B	37.8	74.1	74.1	91.6	83.3
GPT2-XL	35.7	69.6	71.3	90.7	79.3
DeepScaleR-1.5B	54.0	77.0	69.3	91.0	86.0

Further, we measure the language modeling performance using perplexity (PPL) with UIM on different test datasets (Sec. 4.3), including WikiText-103 which the module is trained on, as well as BookCorpus [Zhu et al., 2015] and OpenWebText [Gokaslan and Cohen, 2019] with the same merge module without re-training. To quantify compression, we report token reduction (TR, %), which reflects the reduction in sequence length and KV cache usage.

Finally, we assess the **generalization** ability of UIM on downstream tasks including question answering (QA), document summarization, and math reasoning (Sec. 4.4). In the **task-agnostic** setting, a merge module trained on WikiText-103 is directly applied to all downstream tasks without task-specific supervision or fine-tuning, evaluating its transfer across unseen domains. We further

¹<https://huggingface.co/agentica-org/DeepScaleR-1.5B-Preview>.

Table 2: **Language modeling performance under token reduction.** We report perplexity (PPL; lower is better) and token reduction (TR, %; higher means more tokens are merged). Original denotes the base model.

Backbone	Method	WikiText-103		BookCorpus		OpenWebText		PPL Avg.
		PPL	TR (%)	PPL	TR (%)	PPL	TR (%)	
Llama-3-8B	Original	13.4	–	15.3	–	9.2	–	
	SelectContext	65.0	36.0	46.4	14.0	18.5	13.0	43.3
	LLMLingua-2	76.5	36.0	75.5	14.0	22.9	13.0	58.3
	Merge Module	13.8	36.1	15.3	14.8	9.8	13.8	13.0
Llama-3.2-1B	Original	20.1	–	21.2	–	13.4	–	
	SelectContext	130.4	36.0	67.0	14.0	30.7	13.0	76.0
	LLMLingua-2	129.0	36.0	101.8	14.0	37.3	13.0	89.4
	Merge Module	23.4	36.1	22.0	14.8	14.7	13.8	20.7
GPT2-XL	Original	28.6	–	27.1	–	13.1	–	
	SelectContext	202.5	35.1	93.4	14.0	39.2	12.0	111.7
	LLMLingua-2	250.3	35.1	124.2	14.0	41.5	12.0	138.7
	Merge Module	37.9	35.1	30.0	14.1	14.4	12.5	27.4

study a **downstream adaptation** setting, where the pretrained merge module is specialized to target tasks through task-specific merge-rule construction, fine-tuning, and preference optimization, while keeping the backbone language model frozen. We compare with representative context reduction and prompt compression baselines, including Select Context [Li et al., 2023], LlmLingua [Jiang et al., 2023], LlmLingua2 [Pan et al., 2024], H2O [Zhang et al., 2023], StreamingLLM [Xiao et al., 2024], CPC [Liskavets et al., 2024], and EHPC [Fei et al., 2025].

4.2 Predictive Distribution Alignment

Table 1 reports predictive distribution alignment results on the corresponding *test* split after training. Higher values indicate better agreement between the predictive distributions produced after token merging and those of the original token-based LLM at aligned positions. Across all backbones, UMIM achieves strong alignment scores while reducing the effective sequence length by 35.7%–54.0%. These results validate that the proposed merge module can preserve much of the original next-token behavior using only static input embeddings, providing a strong foundation for downstream compression and generation.

4.3 Language Modeling and Generation

Table 2 reports language modeling performance. All merge modules are trained on the WikiText-103 training split, and the merge rules are derived from the same corpus. Overall, UMIM preserves language modeling behavior across both in-domain and out-of-domain data. Llama-3.1-8B remains close to the original token-based model across all three datasets while achieving substantial token reduction, and Llama-3.2-1.5B shows the same trend with slightly larger degradation. GPT2-XL is the main exception, exhibiting a larger PPL increase after merging, likely due to its weaker backbone and smaller merge module; nevertheless, it remains stronger than external prompt-compression baselines at comparable compression rates. In terms of compression, the WikiText-103 test split, which matches the training domain of the merge module, naturally exhibits the highest token reduction, with nearly 40% of tokens removed. More importantly, UMIM still achieves non-trivial compression on unseen domains, yielding 12–15% token reduction on BookCorpus and OpenWebText without any retraining. This result suggests that the learned merge operator generalizes beyond the training corpus not only in predictive behavior, but also in its ability to induce effective sequence compression.

We also provide qualitative generation examples in Table 3, using greedy decoding with Llama 3.1 8B. In several cases, token merging leaves the generated output unchanged. In others, the output differs slightly from the original model, but remains coherent and semantically plausible. These examples complement the quantitative PPL results by showing that UMIM can reduce the effective input length while largely preserving generation quality. More examples are provided in Appendix H.

Table 3: **Qualitative generations with dynamic token-span merging.** We compare continuations produced by the base model and by UMIM under the same prompt. Bold bracketed spans indicate token spans that are merged into a single surrogate embedding, either in the prompt or during autoregressive generation.

Prompt	Base Model	UMIM
Traditional Chinese literary criticism emphasized [the life of the] author when interpreting a work	The author’s life was seen as the key to understanding the work.	The author’s life [was seen as] the key [to understanding the work] .
In the spring, [his youngest son] , Ian, was born.	He was a beautiful baby boy, and he was the light of his father’s life.	He [was a beautiful baby] , [with a head of thick] , dark [hair and] a round, chubby face.

4.4 Task-Agnostic Generalization to Downstream Tasks

QA and Summarization With the **same** merge modules trained on WikiText-103, we directly apply them to downstream QA and summarization tasks *without any task-specific tuning*. We evaluate zero-shot QA on PIQA [Bisk et al., 2020], Copa [Gordon et al., 2012], OpenBookQA [Mihaylov et al., 2018], ARC_Easy, and ARC_Challenge [Clark et al., 2018], and summarization on CNN/DailyMail [Nallapati et al., 2016] using Rouge [Ganesan, 2018] and BERTScore [Zhang et al., 2020]. For fair comparison, the token reduction (TR) of all baselines is matched to that of our method. Additional results, including code generation, are provided in Appendix H. Across QA benchmarks, UMIM consistently provides the strongest or most robust accuracy–compression trade-off among the compared methods. This holds across all three backbones and across tasks entirely unseen during training, showing that the same pre-trained merge module can transfer directly to unseen downstream tasks without task-specific tuning. A similar pattern is observed on CNN/DailyMail summarization. Under the same controlled token reduction, UMIM remains substantially closer to the original model than all compared baselines, achieving the best Rouge and BERTScore among compressed variants. This further suggests that the learned merge operator preserves generation quality beyond classification-style QA tasks.

Table 4: Performance of UMIM against compression baselines. Results are reported as correct/total problems. TR (%) measures tokens removed.

Method	AIME	TR(%)	AMC	TR(%)
DeepScaleR	15/30	–	37/40	–
UMIM	13/30	40.0	32/40	42.4
SelectContext	8/30	40.0	21/40	42.4
LLMLingua2	11/30	40.0	28/40	42.4

Advanced Math Reasoning We further evaluate UMIM on the AIME 2024 [Mathematical Association of America, 2024] and, we allow up to 15K generation tokens and evaluate pass@k accuracy under extended chain-of-thought decoding. As shown in Table 4 AMC mathematics benchmarks using DeepScaleR, where the merge module is trained on collected long chain-of-thought reasoning trajectories. To specifically test UMIM in a **long-form reasoning setting**, UMIM remains clearly more robust than Select Context and LLMLingua2 under more aggressive compression, retaining strong reasoning performance at over 40% token reduction. Together with the QA and summarization results, this shows that the same pretrained merge operator transfers effectively beyond its training objective, including challenging long-form generation settings with substantial reasoning depth.

4.5 Task-Specific Adaptation of a Pretrained Merge Module

We have shown that a merge module trained on WikiText-103 transfers well to unseen downstream tasks. We denote this module WT. The module itself is general, but the WikiText-derived merge rules are not equally well matched to every target task. This motivates adapting the rule set and the merge module to the target task. We compare five settings on HellaSwag: Task-trained, which trains a merge module from scratch with HellaSwag-specific rules; WT + Rules, which keeps WT fixed and only replaces the merge rules; WT + FT, which further fine-tunes the merge module; WT + DPO, which applies Direct Preference Optimization (DPO) directly without supervised fine-tuning first; and Task-Adapted, our full route defined as WT + Rules + FT + DPO. In all cases, task-specific merge rules are constructed using only the HellaSwag training split, not the validation or test splits.

Table 5: QA task performance of our Merge Module against compression baselines across three backbones. Acc (%) measures accuracy on each benchmark; TR (%) measures the percentage of tokens removed from the input. Bold denotes the best result among compression methods.

Methods	PIQA		Copa		OpenBookQA		ARC-Easy		ARC-Challenge		Avg (%)
	Acc (%)	TR (%)	Acc (%)	TR (%)	Acc (%)	TR (%)	Acc (%)	TR (%)	Acc (%)	TR (%)	
Llama-3.1-8B	79.6	–	76.8	–	43.6	–	76.5	–	49.5	–	65.2
Merge Module	78.4	10	76.8	6	42.4	10	73.7	14	44.8	14	63.2
Select Context	73.3	10	72.6	6	40.8	10	57.2	14	43.5	14	53.5
LLMLingua-2	75.1	10	72.2	6	32.8	10	44.0	14	32.8	14	51.4
CPC	68.7	10	73.4	6	32.0	10	42.5	14	33.8	14	50.1
EHPC	72.7	10	64.6	6	28.2	10	38.3	14	28.8	14	46.5
Llama-3.2-1B	75.5	–	70.6	–	36.0	–	58.8	–	35.5	–	56.3
Merge Module	72.2	10	68.8	6	31.2	10	55.3	14	33.4	14	52.3
Select Context	68.8	10	65.8	6	35.0	10	46.5	14	32.8	14	49.7
LLMLingua-2	69.1	10	69.2	6	31.2	10	37.9	14	26.4	14	46.8
CPC	52.7	10	68.6	6	29.4	10	53.2	14	31.4	14	47.1
EHPC	69.4	10	60.4	6	27.0	10	35.8	14	25.1	14	43.5
GPT2-XL	70.3	–	65.2	–	28.0	–	47.7	–	29.8	–	48.2
Merge Module	68.8	9	62.0	9	29.0	10	43.3	14	26.8	14	46.0
Select Context	64.8	9	61.0	9	29.0	10	40.9	14	25.4	14	44.2
LLMLingua-2	66.6	9	63.8	9	26.4	10	30.7	14	25.8	14	42.6
CPC	63.2	9	61.8	9	28.0	10	34.6	14	27.8	14	43.1
EHPC	65.7	9	57.0	9	25.8	10	32.8	14	20.1	14	40.3

Table 6: CNN/DailyMail summarization performance of our Merge Module against compression baselines on Llama-3.1-8B. TR (%) measures the percentage of tokens removed from the input. R-1, R-2, R-L, and R-Lsum denote ROUGE-1, ROUGE-2, ROUGE-L, and ROUGE-Lsum scores respectively; BERTScore measures semantic similarity. All metrics are higher-is-better. Bold denotes the best result among compression methods.

Method	TR (%)	R-1	R-2	R-L	R-Lsum	Avg	BERTScore
Llama-3.1-8B	–	37.44	15.56	24.31	31.29	27.18	86.76
UMIM	16	35.54	13.86	23.06	29.69	25.54	86.44
SelectContext	16	33.53	11.20	21.43	27.55	23.43	85.08
LLMLingua	16	33.55	10.31	21.33	27.66	23.21	85.33
LLMLingua2	16	33.04	11.32	22.98	28.75	24.02	84.96
StreamingLLM	16	28.81	10.06	18.91	24.29	20.52	84.47
H2O	16	24.42	9.28	16.29	21.24	17.81	82.73

Figure 3 shows a clear progression. Replacing WikiText-derived rules with HellaSwag-specific rules increases merge coverage from about 5% to 12%–43% while maintaining competitive accuracy. Fine-tuning the WT module brings stable improvements across merge rates, showing that WT provides a strong initialization for downstream adaptation. Direct DPO is less stable, especially under heavier compression, whereas Task-Adapted achieves the best accuracy–compression trade-off and exceeds the uncompressed baseline over a broad range of merge rates. Figure 3 further shows that Task-Adapted outperforms SelectContext and LLMLingua-2 at comparable merge rates. The same pattern extends beyond HellaSwag: on ARC-Easy and ARC-Challenge, WT already performs competitively, and Task-Adapted further improves both accuracy and compression (Figure 5).

4.6 Ablation Study and Efficiency Analysis

We conduct ablations along two dimensions: merge-rule construction and merge-module design. Overall, the results show that UMIM is not highly sensitive to a single rule-construction strategy, but it is sensitive to overly aggressive merging. They also show that the current merge module achieves a strong accuracy–compression trade-off under a lightweight parameter budget, with limited gains from further increasing capacity. For merge rules (in Appendix J), we study different frequency thresholds, span construction strategies, and alternative rule-construction methods. The results show that setting the frequency threshold τ too low introduces many low-quality merges and leads to

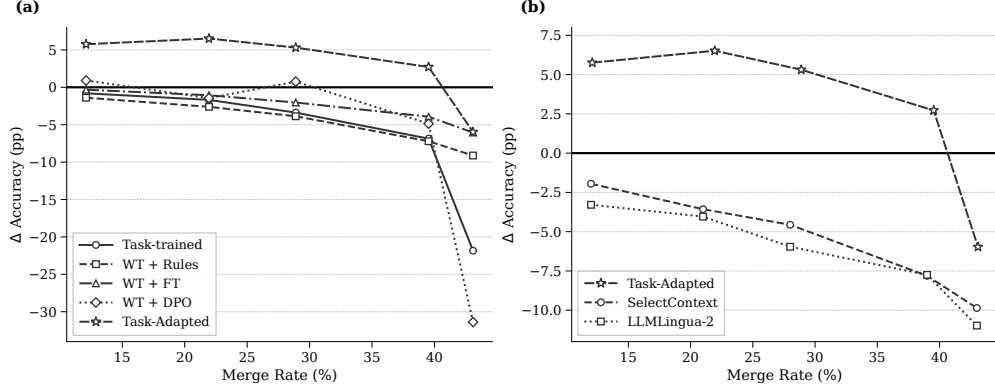


Figure 3: **Task-specific adaptation on HellaSwag.** The y-axis reports accuracy change relative to the uncompressed model (0 = baseline). (a) We compare five adaptation settings, four of which build on the WikiText-103 pretrained module (WT); Task-Adapted (WT + Rules + FT + DPO) achieves the best accuracy–compression trade-off. (b) The strongest adapted variant also consistently outperforms external baselines at comparable merge rates.

clear performance degradation. This suggests that avoiding over-merging is more important than relying on a particular rule heuristic. Across the evaluated rule-construction strategies, we do not observe large performance differences, indicating that UMIM is reasonably robust to the exact rule design as long as the resulting merges provide sufficient training signal and do not over-compress the input. For the merge module (in Appendix J), we compare different parameter sizes and structural configurations. The current attention-based design already performs well with a small parameter budget, while increasing capacity yields only marginal improvements. We also find that simpler linear variants or deeper but less efficient structures do not provide better trade-offs. These results suggest that the current module design is effective for learning surrogate span representations, and that UMIM does not primarily rely on brute-force scaling to achieve strong compression performance.

Following the evaluation protocol of Zip2Zip [Geng et al., 2025], we conduct an end-to-end throughput study on Llama-3.1-8B with a fixed 50% merge ratio. We report tokens/sec for both prefill and decoding, with each setting in Table 13 denoted as `prefill length + decode length`. As shown in Table 13, UMIM provides substantial prefill speedups, with larger gains at longer input contexts because merging directly reduces the effective prompt length. Decoding throughput also improves consistently across settings. These results show that UMIM improves end-to-end inference efficiency in both prompt encoding and autoregressive generation.

5 Conclusion

We have introduced UMIM, a lightweight input-level merging framework that compresses multi-token computation in pretrained language models through learned surrogate embeddings. Operating only on static input embeddings while keeping the backbone model frozen, UMIM reduces effective sequence length and KV-cache usage without modifying the model architecture. Across predictive distribution alignment, language modeling, and downstream evaluations, UMIM preserves the original model behavior while providing substantial compression. A merge module trained once on generic text transfers directly to unseen downstream tasks, demonstrating task-agnostic generalization. We have further shown that task-specific merge rules and merge-module-only adaptation, including finetuning and DPO, can substantially improve the accuracy-compression trade-off while keeping the backbone language model frozen. In some settings, this adapted module even surpasses the original uncompressed baseline. Overall, these results suggest that input-level token merging provides a practical, reusable, and adaptable mechanism for compressing frozen language models.

Limitations

Our current study focuses on establishing UMIM as a practical input-level compression framework for frozen language models across representative models and tasks. Future work may explore more

adaptive merging strategies, broader model scales and domains, and the interaction between pretrained merge modules and downstream adaptation.

References

- Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant J Nair, Ilya Soloveychik, and Purushotham Kamath. Keyformer: Kv cache reduction through key tokens selection for efficient generative inference. *Proceedings of Machine Learning and Systems*, 6:114–127, 2024.
- Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439, 2020.
- Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your vit but faster. In *ICLR*, 2023.
- Yu-Neng Chuang, Tianwei Xing, Chia-Yuan Chang, Zirui Liu, Xun Chen, and Xia Hu. Learning to compress prompt in natural language formats, 2024. URL <https://arxiv.org/abs/2402.18700>.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge, 2018. URL <https://arxiv.org/abs/1803.05457>.
- Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.
- Weizhi Fei, Xueyan Niu, Guoqing Xie, Yingqing Liu, Bo Bai, and Wei Han. Efficient prompt compression with evaluator heads for long-context transformer inference, 2025. URL <https://arxiv.org/abs/2501.12959>.
- Kavita Ganesan. Rouge 2.0: Updated and improved measures for evaluation of summarization tasks, 2018. URL <https://arxiv.org/abs/1803.01937>.
- Tao Ge, Hu Jing, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *The Twelfth International Conference on Learning Representations*, 2024.
- Saibo Geng, Nathan Ranchin, Yunzhen yao, Maxime Peyrard, Chris Wendler, Michael Gastpar, and Robert West. zip2zip: Inference-time adaptive tokenization via online compression, 2025. URL <https://arxiv.org/abs/2506.01084>.
- Aran Gokaslan and Victoria Cohen. OpenWebText Corpus. <http://Skylion007.github.io/OpenWebTextCorpus>, 2019. Accessed: 2025-05-16.
- Andrew Gordon, Zornitsa Kozareva, and Melissa Roemmele. SemEval-2012 task 7: Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In Eneko Agirre, Johan Bos, Mona Diab, Suresh Manandhar, Yuval Marton, and Deniz Yuret, editors, **SEM 2012: The First Joint Conference on Lexical and Computational Semantics – Volume 1: Proceedings of the main conference and the shared task, and Volume 2: Proceedings of the Sixth International Workshop on Semantic Evaluation (SemEval 2012)*, pages 394–398, Montréal, Canada, 7-8 June 2012. Association for Computational Linguistics. URL <https://aclanthology.org/S12-1052/>.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego

Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shao-liang Nie, Sharan Narang, Sharath Rapparthi, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vitor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippou Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Sweet, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim

- Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Rutu Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojuan Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Jitai Hao, Yuke Zhu, Tian Wang, Jun Yu, Xin Xin, Bo Zheng, Zhaochun Ren, and Sheng Guo. Omnikv: Dynamic context selection for efficient long-context llms. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmllingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*, 2023.
- Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677, 2024.
- Julie Kallini, Shikhar Murty, Christopher D Manning, Christopher Potts, and Róbert Csordás. Mrt5: Dynamic token merging for efficient byte-level language models. *arXiv preprint arXiv:2410.20771*, 2024.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- Tian Lan, Deng Cai, Yan Wang, Heyan Huang, and Xian-Ling Mao. Copy is all you need. *arXiv preprint arXiv:2307.06962*, 2023.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.
- Yanhong Li, Zixuan Lan, and Jiawei Zhou. Text or pixels? evaluating efficiency and understanding of LLMs with visual text inputs. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 10564–10578, Suzhou, China, November 2025a. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.558. URL <https://aclanthology.org/2025.findings-emnlp.558/>.

- Yanhong Li, Karen Livescu, and Jiawei Zhou. Chunk-distilled language modeling. In *The Thirteenth International Conference on Learning Representations*, 2025b.
- Yucheng Li. Unlocking context constraints of llms: Enhancing context efficiency of llms with self-information-based content filtering. *arXiv preprint arXiv:2304.12102*, 2023.
- Yucheng Li, Bo Dong, Chenghua Lin, and Frank Guerin. Compressing context to enhance inference efficiency of large language models. *arXiv preprint arXiv:2310.06201*, 2023.
- Zongqian Li, Yixuan Su, and Nigel Collier. 500xcompressor: Generalized prompt compression for large language models, 2024. URL <https://arxiv.org/abs/2408.03094>.
- Barys Liskavets, Maxim Ushakov, Shuvendu Roy, Mark Klibanov, Ali Etemad, and Shane Luke. Prompt compression with context-aware sentence encoding for fast and improved llm inference, 2024. URL <https://arxiv.org/abs/2409.01227>.
- Alisa Liu, Jonathan Hayase, Valentin Hofmann, Sewoong Oh, Noah A Smith, and Yejin Choi. Superbpe: Space travel for language models. *arXiv preprint arXiv:2503.13423*, 2025.
- Mathematical Association of America. American Invitational Mathematics Examination (AIME) 2024. <https://maa.org/math-competitions/american-invitational-mathematics-examination-aime>, February 2024. Accessed: 2025-05-16.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016. URL <https://arxiv.org/abs/1609.07843>.
- Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering, 2018. URL <https://arxiv.org/abs/1809.02789>.
- Jesse Mu, Xiang Li, and Noah Goodman. Learning to compress prompts with gist tokens. *Advances in Neural Information Processing Systems*, 36:19327–19352, 2023.
- Ramesh Nallapati, Bowen Zhou, Cicero dos Santos, Çağlar Gulçehre, and Bing Xiang. Abstractive text summarization using sequence-to-sequence RNNs and beyond. In Stefan Riezler and Yoav Goldberg, editors, *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, pages 280–290, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/K16-1028. URL <https://aclanthology.org/K16-1028/>.
- Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, et al. Byte latent transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*, 2024.
- Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. Llm1ingua-2: Data distillation for efficient and faithful task-agnostic prompt compression, 2024. URL <https://arxiv.org/abs/2403.12968>.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Michael Ryoo, AJ Piergiovanni, Anurag Arnab, Mostafa Dehghani, and Anelia Angelova. Token-learner: Adaptive space-time tokenization for videos. *Advances in neural information processing systems*, 34:12786–12797, 2021.
- Ninglu Shao, Shitao Xiao, Zheng Liu, and Peitian Zhang. Flexibly scaling large language models contexts through extensible tokenization. *arXiv preprint arXiv:2401.07793*, 2024.
- Jaekang Shin, Myeonggu Kang, Yunki Han, Junyoung Park, and Lee-Sup Kim. Atom: Adaptive token merging for efficient acceleration of vision transformer. *IEEE Transactions on Computers*, 2025.

- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024.
- Fangyuan Xu, Weijia Shi, and Eunsol Choi. Recomp: Improving retrieval-augmented lms with context compression and selective augmentation. In *The Twelfth International Conference on Learning Representations*, 2024.
- Zhentao Xu, Fengyi Li, Albert Chen, and Xiaofeng Wang. Procut: Llm prompt compression via attribution estimation, 2025. URL <https://arxiv.org/abs/2508.02053>.
- Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. Byt5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022.
- Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. Megabyte: Predicting million-byte sequences with multiscale transformers. *Advances in Neural Information Processing Systems*, 36:78808–78823, 2023.
- Xin Yuan, Hongliang Fei, and Jinoo Baek. Efficient transformer adaptation with soft token merging. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3658–3668, 2024.
- Ivan Zakazov, Berke Argin, Oussama Gabouj, Kamel Charaf, Alexander Sharipov, Alexi Semiz, Lorenzo Drudi, Nicolas Baldwin, and Robert West. Cmprsr: Abstractive token-level question-agnostic prompt compressor, 2026. URL <https://arxiv.org/abs/2511.12281>.
- Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert, 2020. URL <https://arxiv.org/abs/1904.09675>.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710, 2023.
- Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *Proceedings of the IEEE international conference on computer vision*, pages 19–27, 2015.

A Empirical Study

A.1 Empirical Summary

We will subsequently provide additional content to supplement the main text in the following sections for UMIM. The central question of this work is whether the sequential computation of multiple adjacent tokens in a frozen autoregressive language model can be approximated by a single learned input-level representation. This is different from token deletion, prompt rewriting, hidden-state pruning, or introducing a new tokenizer. UMIM keeps the pretrained language model unchanged: the tokenizer, vocabulary, model architecture, and backbone parameters are all fixed. Only a lightweight merge module is trained to map selected token spans into single surrogate embeddings.

The main empirical message of the paper is that input-level token-span merging can substantially reduce the effective sequence length while preserving the behavior of the original model to a large extent. The merge module operates before Transformer computation, using only static token embeddings as input. After training, the resulting surrogate embedding can replace a short multi-token span as a single computational unit. This allows the model to process compressed prompts and compressed decoding contexts without changing the underlying language model. In this sense, UMIM is not merely a heuristic for shortening text; it is a learned input-level approximation of multi-step token computation in frozen Transformer language models.

A key strength of UMIM is that the compression is achieved with strong performance preservation. Across language modeling and downstream tasks, the compressed model remains close to the uncompressed base model while reducing the effective sequence length. This is important because many efficient inference methods obtain speed or memory savings by deleting tokens, dropping context, or compressing information in a task-dependent way, often causing a visible accuracy degradation. UMIM instead attempts to preserve the functional role of the merged span: the span is still represented, but its multi-step computation is replaced by a single learned surrogate embedding. This design explains why UMIM can reduce sequence length and KV-cache usage while maintaining strong predictive and downstream performance.

Another important finding is that the merge module trained on a general corpus can transfer directly to downstream tasks. In the task-agnostic setting, the UMIM module trained on WikiText-103 is applied to unseen tasks without updating the backbone language model and without task-specific supervision. Even under this direct-transfer setting, UMIM achieves a strong accuracy-compression trade-off and consistently outperforms representative prompt-compression and context-reduction baselines at comparable token reduction rates. This result suggests that the learned module is not simply memorizing a fixed training corpus; rather, it learns a reusable span-to-surrogate mapping that can be consumed by frozen language models across different tasks.

The distinction between merge rules and the merge module is central to the method. Frequency-based rules decide where merging is attempted, but they do not define the compressed representation itself. The compressed representation is produced by the learned merge module. Therefore, UMIM should not be understood as only an n-gram compression heuristic. The rules provide stable and recurring candidate spans, while predictive distillation teaches the module how to replace the original multi-token computation with a single embedding that preserves the language model’s output behavior. This separation makes the framework both reusable and adaptable: the pretrained module can be reused across domains, while the rule set can be reconstructed when a target task requires higher merge coverage.

The task-specific adaptation experiments further demonstrate this point. Starting from the WikiText-trained merge module, we can rebuild task-specific merge rules and then adapt only the merge module through lightweight fine-tuning and preference optimization. The backbone language model remains frozen throughout this process. This setting is substantially more practical than retraining or modifying the full language model. More importantly, the adapted module can achieve a stronger accuracy-compression trade-off than both the directly transferred module and external compression baselines. In some settings, the adapted UMIM variant can preserve or even exceed the accuracy of the original uncompressed base model while still reducing the effective sequence length by a large margin. This indicates that UMIM is not only a compression method, but also a practical framework for learning reusable and adaptable input-level approximations of sequential computation.

A.2 LLM Background

Modern language models generate text in an autoregressive manner, where tokens are processed and generated sequentially. This sequential process leads to high inference cost, especially when the input or generated sequence becomes long. Motivated by this practical limitation, we explore whether multiple tokens can be represented by a single learned embedding during inference. The goal is to reduce token-level computation while preserving the model’s original predictive behavior as much as possible.

In the following subsections, we describe our early attempts and empirical observations, which may provide valuable experience.

A.3 Early Attempt One: Embedding Similarity-Based Token Merging

In the early stages of this work, we explored token merging from a purely representation-based and linguistically motivated perspective. Since our goal was to reduce redundant token-level computation without introducing additional training or modifying the language model, a natural first attempt was to identify semantically similar tokens and merge them directly at the embedding level.

Concretely, given a token sequence (x_1, x_2, x_3, x_4) , we considered computing the cosine similarity between adjacent token embeddings, such as $e(x_2)$ and $e(x_3)$. If the similarity exceeded a predefined threshold, the two tokens were regarded as semantically redundant and merged into a single representation, typically by simple averaging of their embeddings. This approach was motivated by the intuition that semantically similar tokens may play interchangeable roles in language understanding, and that their embeddings—learned by large language models—should reflect this similarity.

Importantly, this design deliberately avoided any form of training or supervision. The merging decision relied solely on static token embeddings and geometric similarity measures, making it lightweight and model-agnostic. From a linguistic and representation-learning standpoint, this approach appears reasonable and aligns with common practices in embedding-based clustering and semantic similarity analysis.

However, empirical results quickly revealed that this strategy failed to preserve model behavior in practice. Even when merging token pairs with relatively high cosine similarity, the resulting generations often deviated noticeably from the original outputs. In some cases, merging led to abrupt changes in the predicted next-token distribution, indicating that semantic proximity in embedding space was insufficient to guarantee computational substitutability within the language model.

Upon further analysis, we identified several fundamental limitations of this approach. First, cosine similarity in high-dimensional embedding spaces lacks a stable and transferable threshold. The similarity distribution is highly concentrated, making it difficult to distinguish genuinely mergeable token pairs from unrelated ones in a robust manner. Small variations in threshold choice often resulted in either excessive merging or negligible compression.

More fundamentally, semantic similarity between token embeddings does not imply equivalence in their computational roles within a Transformer. Token embeddings serve only as inputs to a highly nonlinear and context-dependent computation pipeline involving positional encodings, multi-head attention, and deep feed-forward transformations. Two tokens that appear similar in embedding space may interact very differently with surrounding context, attend to different positions, or trigger distinct internal computation patterns. As a result, replacing multiple tokens with an averaged embedding introduces a distribution shift that the model was never trained to handle.

A.4 Early Attempt 2: Training-Based Pairwise Token Merging

After observing that purely heuristic or geometry-based token merging strategies fail to reliably preserve model behavior, we naturally shifted to a training-based perspective. From this viewpoint, it is reasonable to hypothesize that there exists a learnable *super embedding* in the embedding space that can replace multiple token embeddings and approximate their joint computational effect. The theoretical motivation behind this assumption will be discussed in later sections.

In this stage, we began with the simplest and most controlled setting: *pairwise token merging*. Given an input sequence (x_1, x_2, x_3, x_4) , we deterministically merged (x_1, x_2) into a single token and (x_3, x_4) into another, thereby reducing the sequence length by half while keeping the merging rule

fixed. This design avoids the complexity of dynamic span selection and allows us to isolate the learnability of the merging operation itself.

As an initial sanity check, we froze a GPT-2 model and trained only a lightweight merging module on a small number of sampled sentences. Even under this minimal setup, we observed that the merged representations could approximate the original next-token predictive distributions after a few training steps. This result indicates that, in principle, such super embeddings do exist and can be learned under constrained conditions.

We then extended this experiment to a more realistic setting using the IMDB dataset. In this configuration, input sequences of length 512 were uniformly compressed via pairwise merging into 256 tokens. At this scale, the problem shifted from feasibility to stability and generalization. We found that the expressive capacity of the merging module became a critical factor. To achieve satisfactory distribution alignment with the frozen GPT-2 model, the merging module required substantially increased capacity. Consequently, we employed a deep, over-parameterized linear network with nonlinear activations, LayerNorm, and residual connections to transform the merged embeddings back to the original embedding dimension. While this design lacks theoretical elegance, it represented the most effective empirical solution under the constraints of this experimental setup.

Regarding the training objective, we used distribution-level alignment metrics consistent with those in the main experiments (e.g., Top-1 and Top- p overlap). A notable empirical observation was that using KL divergence alone for distillation often resulted in unstable training or poor convergence. In this setting, the merging module was required to directly approximate the predictive distribution induced by two consecutive decoding steps, leading to noisy and weak optimization signals. To mitigate this issue, we introduced an additional cosine similarity constraint at the embedding or sentence level, providing a geometric regularization that significantly improved training stability.

It is important to emphasize that the design choices in this stage—including fixed pairwise merging, a highly over-parameterized merging network, and the combination of KL-based distillation with cosine regularization—were driven primarily by empirical considerations rather than theoretical optimality. Although this approach can yield reasonable results in controlled and moderate-scale settings, its reliance on fixed segmentation and excessive parameterization exposes clear limitations in scalability and generalization. These observations motivated the search for a more general and structurally grounded merging formulation, which we present in the subsequent sections.

A.5 Foundation to our current Work

After extensive exploration and empirical refinement, we arrive at the current merging formulation. The concrete architectural and implementation details have already been described thoroughly in the main text and appendices. In this section, we focus instead on a more fundamental question: *why this approach is expected to work in principle*. Our goal is to clarify that the method is not a heuristic construction, but rather the result of parallel theoretical reasoning and engineering validation.

We begin with the construction of the merge set. Any token merging strategy must first address a basic question: *which token pairs or spans should be merged*. This question is closely tied to the tokenization mechanisms employed by modern language models.

Most contemporary language models rely on Byte Pair Encoding (BPE) or its variants for tokenization. BPE is not designed to recover linguistically complete units; instead, it is a statistically driven compression algorithm. Starting from atomic units (characters or bytes), BPE repeatedly merges the most frequent adjacent token pairs in the training corpus, aiming to reduce sequence length while maintaining a compact vocabulary. As a result, the segmentation produced by BPE is governed by frequency statistics rather than semantic or linguistic completeness.

While highly effective in practice, this process often leads to the fragmentation of expressions that humans naturally perceive as single semantic units. For example, adverbs such as *excitedly* may be decomposed into *excited* and *ly*, and common multi-word expressions such as *machine learning* or *love to do something* are represented as multiple tokens. From a linguistic or cognitive perspective, these expressions typically function as indivisible units, yet they are split due to the statistical nature of the tokenizer.

Our approach does not seek to revise or replace BPE, but rather to complement it at inference time. Specifically, we ask whether frequently occurring composite expressions that are split by the

tokenizer can be re-aggregated into larger units during model execution. This motivates the use of high-frequency n -grams as candidates for merging. Such a choice can be interpreted both as compensating for the side effects of BPE segmentation and as explicitly modeling structures that the language model has already encountered repeatedly during training.

Importantly, the use of high-frequency n -grams is not a purely heuristic decision. From a learning-theoretic perspective, frequency is closely tied to learnability. High-frequency n -grams correspond to input patterns that appear many times in the training corpus, and thus participate in a large number of gradient updates. As a consequence, the language model tends to exhibit more stable and consistent behavior when processing these token spans. Compared to rare or idiosyncratic spans, frequent n -grams are associated with better-conditioned empirical estimates and lower variance in learned representations, making them more suitable targets for reliable approximation.

This observation leads naturally to the question of whether a single embedding can represent an entire n -gram at the input level. From the outset, we hypothesize the existence of such a *super token* or *super embedding*. This assumption follows directly from the general principles of model training. In practice, learning systems approximate an underlying data-generating distribution that cannot be accessed explicitly. Instead, training proceeds by fitting parametric models to empirical observations or to surrogate distributions induced by teacher models.

In our setting, a pre-trained and frozen language model defines a stable and observable mapping from input token embeddings to predictive distributions. For high-frequency n -grams, this mapping has already been learned repeatedly across diverse contexts during pre-training. Training a merging module does not introduce new semantic content; rather, it seeks to approximate an existing, implicitly learned computation by re-parameterizing it at the embedding level. From a function approximation perspective, if the behavior induced by a multi-token span is stable and consistently observable, then it is reasonable to expect that a parameterized module can learn a compact representation that preserves this functional role.

Viewed through this lens, the proposed merging formulation does not invent new linguistic units, nor does it impose arbitrary structural assumptions. Instead, it explicitly captures composite computational patterns that are already encoded within the language model. The resulting super embeddings serve as compressed representations of these patterns, enabling token reduction while preserving the model’s predictive behavior.

A.6 Supplementary Clarifications

This subsection provides additional clarification on why our method restricts merging candidates to n -grams of length 2–4. We emphasize that this design choice reflects a statistical and practical prior rather than a fundamental limitation of the proposed formulation.

Our merge module is trained on WikiText-103, a widely used benchmark corpus in natural language processing. The choice of this dataset is motivated by its representativeness and accessibility, while the scale of training is constrained by available computational resources. Within this setting, we carefully considered the range of n -grams to retain for training and inference, ultimately focusing on spans of length 2 to 4.

A key constraint arises from the training procedure itself. The merge module does not support overlapping merges. For example, given a sequence (x_1, x_2, x_3, x_4) , if (x_2, x_3) is merged into a super token x_{23} , it is not possible to subsequently merge (x_{23}, x_4) into a larger unit. Instead, a span such as (x_2, x_3, x_4) must be merged as a whole from the outset. Supporting overlapping or sequential merges would introduce dynamic, path-dependent merge structures, which prevent efficient batch-level training and would incur prohibitive computational and memory costs. We note that such overlapping merges are theoretically feasible, as discussed in the previous section, but remain impractical under current training constraints.

From a statistical perspective, limiting the merge set to 2–4 grams is also well motivated. Most higher-order n -grams (e.g., $n \geq 5$) contain frequent sub-spans of length 2, 3, or 4. As a result, retaining these shorter spans already captures the majority of repetitive and semantically coherent structures present in the data. Explicitly modeling longer spans yields diminishing returns while significantly increasing training complexity.

Furthermore, the choice of shorter n -grams is advantageous from an inference-efficiency standpoint. During autoregressive decoding, tokens are generated sequentially. Merging a span can only occur once all tokens in that span have been generated. Consequently, merging a 2-gram allows compression to take effect immediately after the second token is produced, reducing the KV cache size and accelerating subsequent decoding steps. In contrast, merging a 5-gram requires waiting until all five tokens are generated, during which time equivalent or greater benefits could already be obtained by merging shorter spans such as a 2-gram followed by a 3-gram. Longer spans therefore delay the onset of computational savings without providing proportional gains.

In principle, the framework can be extended to arbitrarily long n -grams. However, from both a training and inference perspective, such extensions offer limited practical value. The restriction to 2–4 grams represents a balanced trade-off between statistical coverage, training feasibility, and inference-time efficiency, and is not central to the conceptual contribution of our method.

A.7 Conclusion

Having presented the full methodology and empirical analysis, we conclude by clarifying the broader research perspective and contribution of this work. Our goal is not to advocate a single, definitive token merging strategy, but to articulate and validate a new way of thinking about token-level computation in large language models.

Through systematic exploration from the embedding perspective, we demonstrate that, in principle, multiple tokens can be functionally replaced by a single *super token* at the embedding level, without materially altering the model’s predictive distribution. This result does not rely on a particular architectural trick, but follows from the observation that language models implicitly learn stable, compositional behaviors over frequently occurring token spans during training.

The specific merge module and merge set construction presented in this paper should therefore be viewed as one concrete and empirically effective instantiation of this idea, rather than the only possible realization. Our design choices are guided by theoretical considerations, engineering feasibility, and empirical validation, but alternative formulations—such as different merging criteria, architectures, or training regimes—may be equally viable within the same conceptual framework.

Due to computational and resource constraints, our exploration is necessarily limited in scope. Nonetheless, we hope this work serves as a starting point for further investigation. In particular, future research may consider incorporating merging mechanisms directly into model pretraining, or jointly optimizing merge modules alongside backbone models, rather than treating them as post-hoc components.

In this sense, our contribution is not solely a new module, but a research direction: viewing token-level sequential computation as a structure that can be reparameterized, compressed, and approximated. We believe this perspective opens new avenues for improving the efficiency and scalability of language model inference.

B Merging Set Mining Details

Given a text corpus $\mathcal{C}$, we tokenize all documents using the base model’s tokenizer. The resulting token sequence is partitioned into non-overlapping segments of length $L = 512/1024$ for efficient batch processing. In the following, we detail each step of our high-frequency n -gram mining.

N-gram Extraction and Frequency Counting For each sequence, we enumerate all contiguous n -grams for $n \in \{2, 3, 4\}$ and compute their global frequency across the corpus. Let $S = \{s_1, s_2, \dots, s_N\}$ denote the collection of tokenized segments, where $s_i = (t_{i,1}, t_{i,2}, \dots, t_{i,L})$. For each n -gram $g = (t_j, t_{j+1}, \dots, t_{j+n-1})$, we maintain a frequency count $f(g)$.

Threshold-based Candidate Selection We retain only those n -grams whose corpus-wide frequency $f(g)$ exceeds a preset threshold τ (e.g., $\tau = 500, 50, 5$; see ablation study). This yields a high-frequency n -gram set $\mathcal{R} = \{g \mid f(g) \geq \tau\}$ for all $2 \leq n \leq 4$. We did more on the Hellaswag test.

Hierarchical Overlap Filtering Since n -grams may overlap (e.g., a trigram may share tokens with a bigram or fourgram), we apply a two-stage filtering procedure:

1. **Intra-set Filtering:** Within each fixed-length n -gram set, we remove redundant candidates with identical token sequences and keep a single entry with its corpus-wide frequency count.
2. **Inter-set Filtering:** Across n -gram sets of different lengths, we eliminate any lower-order n -gram that is fully contained within a higher-order m -gram ($m > n$) present in $\mathcal{R}$, so that longer and more specific spans are preferred as merge candidates.

Reasons for 2- to 4-grams We restrict our merging set $\mathcal{R}$ to n -grams of length 2 to 4 for both linguistic and computational reasons. Empirically, 2- to 4-gram phrases correspond to the most meaningful word groups in natural language and align with common tokenizer outputs. Including much longer n -grams (e.g., 8-grams or 10-grams) would reduce the practical number of merges, as longer n -grams mask and exclude shorter, high-frequency candidates within their span. Additionally, for autoregressive generation, merging long n -grams is impractical since the model must wait until all tokens of a long span have been generated to determine if a merge is possible, introducing latency and reducing compression effectiveness. Using 2-4 grams, we achieve a balance between merge granularity, linguistic coverage, and computational efficiency.

Different N-grams chunk numbers We present the specific merge rules used in our experiments, with the threshold τ set to 5 (see Section 3.1 and Section 4.1). For Llama-3.1-8B, Llama-3.2-1.5B, and GPT2-XL, the merge rules are derived from Wikitext-103. In contrast, the merge rules for DeepScaleR-1.5B-Preview are selected based on our own generated dataset (see Section F.1).

Table 7: Different N-grams chunk numbers

Model	2-grams	3-grams	4-grams
Llama-3.1-8B	216700	684793	1266080
Llama-3.2-1.5B	216700	684793	1266080
GPT2-XL	218415	702570	1241657
DeepScaleR-1.5B-Preview	35691	212367	1665423

C Module Architecture and Integration

We describe more details of the merge module in Section 3.2 and how it is integrated into the base LM at the input level.

We design the UMIM merge module as a lightweight, attention-based pooling network that operates solely on static input embeddings, prior to any Transformer computation. Its primary function is to serve as a universal replacement for repeated spans of contiguous tokens across diverse contexts, producing a single surrogate embedding that captures the aggregate functional role of the original span. The module supports variable-length token spans (with n ranging from 2 to 4 in our experiments), which are flexibly handled by its multi-head pooling mechanism, and is agnostic to the underlying model architecture.

Formally, let d denote the embedding dimension of the target language model and h the number of attention heads, such that each head has dimension $d_h = d/h$. Given the token span embeddings $e(x_{t:t+n}) \in \mathbb{R}^{d \times n}$, the merge module operates as follows. We use a learnable query parameter $Q \in \mathbb{R}^{h \times d_h}$, where the i -th row $q_i \in \mathbb{R}^{d_h}$ is the query vector for head i . We first apply a shared key-value projection W^{kv} to the span embeddings. For each head i , attention weights are computed via a scaled dot product between q_i and the corresponding head-specific slice of $W^{kv}e(x_{t:t+n})$. The output of each head is then computed as a weighted sum over the same projected span representations. Finally, the outputs from all heads are concatenated and passed through an output projection W^o to obtain the final surrogate embedding:

$$\mathcal{M}_\phi(e(x_{t:t+n})) = W^o[o_1; \dots; o_h]^\top, \quad o_i = \text{softmax}\left(\frac{q_i^\top W_i^{kv} e(x_{t:t+n})}{\sqrt{d_h}}\right) (W_i^{kv} e(x_{t:t+n}))^\top.$$

Here, W_i^{kv} denotes the head-specific slice of the shared projection W^{kv} . The learnable parameters are $\phi = \{Q, W^{kv}, W^o\}$. The total number of parameters is approximately $d + 2d^2$, ignoring the small number of parameters introduced by LayerNorm, which is lightweight relative to the base model.

During training, the merge module is applied to all eligible spans as defined by the high-frequency n -gram merge rule $\mathcal{R}$. Each identified span is replaced by its corresponding surrogate embedding, and the resulting sequence is updated accordingly. The merge operation is performed once, prior to any downstream Transformer layers or autoregressive generation. This design ensures that the merge module can be integrated seamlessly into any Transformer-based language model pipeline, requires minimal overhead, and does not alter the base model parameters or architecture.

In summary, the UMIM merge module provides a flexible, learnable mechanism for compressing repeated multi-token spans into single embeddings, enabling both efficient inference and effective knowledge distillation without modifying the underlying language model structure.

Pseudocode 1: Multi-Head Pooling Module

```
# MultiHeadPooling module
# tokens: (batch, k, d_model)
# num_heads: number of pooling heads
# head_dim: d_model // num_heads
# query: (num_heads, head_dim), learnable queries

tokens_proc = token_processor(tokens)

tokens_proc = tokens_proc.view(batch, k, num_heads, head_dim)
tokens_proc = tokens_proc.permute(0, 2, 1, 3)

query = query.unsqueeze(0).unsqueeze(2)
query = query.expand(batch, num_heads, 1, head_dim)

attn_scores = einsum(
    "bnqd,bnkd->bnk", query, tokens_proc
) / sqrt(head_dim)

attn_weights = softmax(attn_scores, dim=2).unsqueeze(-1)

pooled = sum(tokens_proc * attn_weights, dim=2)

merged = pooled.view(batch, d_model)
output = output_proj(merged)
```

D Multi-Step Distillation Loss and Alignment Metrics

Here we describe in more detail the distillation loss from multi-step computation to compressed input after UMIM merging as introduced in Section 3.3, as well as how the learning algorithm is implemented, and how the loss alignment is evaluated with different metrics.

Masking, Alignment, and Index Construction. Sequence length and target alignment are altered after token merging. To ensure correct supervision and evaluation, we precompute three masks and an index mapping for every input sequence:

- **Attention Mask:** A binary mask of shape $[\text{batch_size}, \text{seq_len}]$, where 1 indicates valid tokens (including merged/retained tokens) and 0 denotes padding. This mask is used for all Transformer operations.
- **Padding Mask:** A mask that distinguishes true tokens from padding after batch collation and zero-padding to a fixed length.
- **Loss Mask:** A binary mask marking tokens that should be counted toward the loss and metric computation. Importantly, this mask is constructed so that:
 - All tokens prior to the first merge are masked out (0).

- Within any merged span, only the *last* (surrogate) token is marked as 1; all others are set to 0 to prevent repeated counting and label misalignment.
- All padding tokens are masked out.
- **Index Mapping:** We store, for each sequence, the mapping from original to compressed token positions, so that teacher and student outputs can be aligned for supervision and metrics.

Before each forward pass and loss computation, we assert that the teacher and student masks are perfectly aligned; if not, an exception is raised to prevent silent misalignment.

Loss Computation. We compute the distillation loss as the average cross-entropy between the predictive distributions of the base LLM (teacher) and the merge module (student), but crucially, the loss is only evaluated at positions directly influenced by the merge process. Specifically, we use a binary mask to select tokens for supervision:

- For all tokens before the first merge, the base LLM is frozen and the outputs of teacher and student are identical; these positions are excluded from loss computation as they provide no meaningful learning signal.
- For each merged span, only the final token is retained in the compressed sequence. The distillation target for each retained token is the original model’s output at that position.
- The loss is normalized by the total number of supervised tokens (the sum of the mask), ensuring the reported value reflects only the positions where the merge module has influence.

Mathematically, for aligned and filtered outputs $p_i^{\text{teacher}}, p_i^{\text{student}}$ at valid positions i , the loss is:

$$\mathcal{L} = -\frac{1}{N_{\text{valid}}} \sum_{i=1}^{N_{\text{valid}}} \sum_v p_i^{\text{teacher}}(v) \log p_i^{\text{student}}(v)$$

where N_{valid} is the total number of masked-in tokens (i.e., $\sum \text{loss_mask}$ over the batch).

Cross-Entropy and KL Divergence: Although distillation is often formulated with the Kullback–Leibler (KL) divergence, in practice, we use cross-entropy for computational efficiency and simplicity. The KL divergence between two distributions P (teacher) and Q (student) is:

$$D_{\text{KL}}(P\|Q) = \sum_v P(v) \log \frac{P(v)}{Q(v)} = H(P, Q) - H(P)$$

where $H(P, Q)$ is the cross-entropy and $H(P)$ is the (teacher) entropy. Crucially, $H(P)$ depends only on the (frozen) teacher and is constant with respect to the student/merge module parameters. Thus, minimizing $D_{\text{KL}}(P\|Q)$ is equivalent (in gradient and optimization) to minimizing the cross-entropy $H(P, Q)$. Therefore, we implement the distillation loss as cross-entropy for simplicity, omitting the constant $H(P)$ term.

This choice does not change the optimization or the learned solution, but avoids unnecessary computation and numerical instability from subtracting nearly equal terms.

Metric Computation and Reporting. We assess model fidelity using several metrics, computed strictly on masked and aligned positions to ensure fair evaluation and to avoid score inflation or deflation due to padding or repeated tokens. The reported metrics are as follows:

- **Top-1 Accuracy:** At each valid position, we measure the proportion of cases where the token with the highest predicted probability (i.e., the greedy argmax) matches between the student and teacher distributions. This metric reflects exact prediction agreement.
- **Top- k Overlap:** For $k = 3$ and $k = 10$, we compute the overlap ratio between the sets of top- k predicted tokens from the teacher and student distributions at each valid position. The overlap is defined as the size of the intersection divided by k , with the final score averaged across all valid positions. This metric quantifies agreement within the highest-probability candidate sets.

- **Top- p Overlap:** For each valid position, we identify the minimal sets of tokens in both the teacher and student distributions whose cumulative probability mass meets or exceeds a threshold p (we report results for $p = 0.9$). The overlap is calculated as the size of the intersection divided by the size of the smaller set. This metric captures the extent to which the models agree on the tokens that collectively account for the majority of the probability mass.

Example Mask Construction. Consider an input sequence: $[x_1, x_2, x_3, x_4, x_5, x_6, \text{pad}, \dots]$, where (x_2, x_3) and (x_5, x_6) are merged. The resulting masks are:

Position	x_1	(x_2, x_3)	x_4	(x_5, x_6)	pad	...
Attention Mask	1	1	1	1	0	...
Loss Mask	0	1	1	1	0	...

Algorithm 1 Masked Cross-Entropy Loss Computation

Require: Compressed probabilities $\text{CP} \in \mathbb{R}^{B \times S \times V}$, original probabilities $\text{TP} \in \mathbb{R}^{B \times S \times V}$, loss mask $M_{\text{loss}} \in \{0, 1\}^{B \times S}$, attention mask $M_{\text{attn}} \in \{0, 1\}^{B \times S}$

Ensure: Average masked cross-entropy loss $\mathcal{L}$

- 1: Initialize aligned teacher list $\mathcal{T}$, aligned student list $\mathcal{S}$, and packed validity mask list $\mathcal{F}$
- 2: For each sample i in the batch:
 - 3: Compute $n_{\text{attn}} \leftarrow \sum_j M_{\text{attn}}[i, j]$
 - 4: Compute $n_{\text{loss}} \leftarrow \sum_j M_{\text{loss}}[i, j]$
 - 5: Initialize $M_{\text{adj}}[i] \leftarrow M_{\text{attn}}[i]$
 - 6: If $n_{\text{attn}} > n_{\text{loss}}$, set the first $n_{\text{attn}} - n_{\text{loss}}$ valid positions of $M_{\text{adj}}[i]$ to 0
 - 7: Select teacher rows: $T_i \leftarrow \text{TP}[i][M_{\text{loss}}[i] = 1]$
 - 8: Select student rows: $S_i \leftarrow \text{CP}[i][M_{\text{adj}}[i] = 1]$
 - 9: Pad T_i and S_i to fixed length L
 - 10: Construct packed validity mask $F_i \in \{0, 1\}^L$, where $F_i[l] = 1$ for selected rows and 0 for padded rows
 - 11: Append T_i , S_i , and F_i to $\mathcal{T}$, $\mathcal{S}$, and $\mathcal{F}$
 - 12: Stack aligned teacher probabilities $T_{\text{aligned}} \in \mathbb{R}^{B \times L \times V}$
 - 13: Stack aligned student probabilities $S_{\text{aligned}} \in \mathbb{R}^{B \times L \times V}$
 - 14: Stack packed validity masks $F \in \{0, 1\}^{B \times L}$
 - 15: Compute masked cross-entropy:

$$\mathcal{L} = -\frac{1}{\sum_{b,l} F[b, l]} \sum_{b,l} F[b, l] \sum_v T_{\text{aligned}}[b, l, v] \cdot \log(S_{\text{aligned}}[b, l, v] + \epsilon)$$

16: **return** $\mathcal{L}$

Algorithm 2 Masked Top-1 Accuracy Computation

Require: Teacher probabilities $T \in \mathbb{R}^{B \times S \times V}$, student probabilities $S \in \mathbb{R}^{B \times S \times V}$, teacher mask $M_T \in \{0, 1\}^{B \times S}$, student mask $M_S \in \{0, 1\}^{B \times S}$

Ensure: Masked top-1 accuracy Acc_{top1}

- 1: For each batch b , pad teacher and student sequences to fixed length L
 - 2: For each sample b :
 - 3: Select rows in $T[b]$ where $M_T[b] = 1$, pad to L ; repeat for $S[b]$ and $M_S[b]$
 - 4: Create mask vectors $F_T[b], F_S[b]$ where 1 marks valid rows
 - 5: Stack all to get $T_{\text{aligned}}, S_{\text{aligned}}, F_T, F_S$
 - 6: Compute top-1 predictions for both: $\text{top1}_T = \text{argmax}_v(T_{\text{aligned}})$, $\text{top1}_S = \text{argmax}_v(S_{\text{aligned}})$
 - 7: Ensure $F_T = F_S$; if not, raise an error
 - 8: Compute combined mask $F = F_T \wedge F_S$
 - 9: Compute match matrix $M = (\text{top1}_T == \text{top1}_S)$
 - 10: numer = $\sum_{b,l} M[b,l] \cdot F[b,l]$
 - 11: denom = $\sum_{b,l} F[b,l]$
 - 12: $\text{Acc}_{\text{top1}} = \begin{cases} 0, & \text{denom} = 0 \\ \text{numer}/\text{denom}, & \text{otherwise} \end{cases}$ **return** Acc_{top1}
-

Algorithm 3 Masked Top- k Overlap Computation

Require: Teacher probabilities $T \in \mathbb{R}^{B \times S \times V}$, student probabilities $S \in \mathbb{R}^{B \times S \times V}$, teacher mask $M_T \in \{0, 1\}^{B \times S}$, student mask $M_S \in \{0, 1\}^{B \times S}$, overlap k

Ensure: Masked top- k overlap Overlap_k

- 1: For each sample, select and pad teacher/student logits as in Algorithm 2; align and flatten valid rows
 - 2: For each valid position m :
 - 3: Compute top- k indices for teacher: $I_m^T = \text{TopK}(T_m, k)$
 - 4: Compute top- k indices for student: $I_m^S = \text{TopK}(S_m, k)$
 - 5: Count overlap: $c_m = |I_m^T \cap I_m^S|$
 - 6: Compute per-position ratio: $r_m = c_m/k$
 - 7: $\text{Overlap}_k = \text{mean}_m(r_m)$ **return** Overlap_k
-

Algorithm 4 Masked Top- p Overlap Computation

Require: Teacher probabilities $T \in \mathbb{R}^{N \times V}$, student probabilities $S \in \mathbb{R}^{N \times V}$, mask $M \in \{0, 1\}^N$, threshold $p \in (0, 1)$

Ensure: Top- p overlap metric Overlap_p

- 1: For all positions i where $M[i] = 1$ (i.e., valid tokens):
 - 2: Sort $T[i]$ in descending order, obtain sorted indices I_i^T , values V_i^T
 - 3: Compute cumulative sum C_i^T of V_i^T
 - 4: Find minimal k_T such that $C_i^T[k_T] \geq p$
 - 5: Define $P_i^T = \{I_i^T[0], \dots, I_i^T[k_T]\}$
 - 6: Repeat above steps for $S[i]$, obtain P_i^S
 - 7: Compute intersection size: $n_{\cap,i} = |P_i^T \cap P_i^S|$
 - 8: Let $n_{\min,i} = \min(|P_i^T|, |P_i^S|)$
 - 9: Set overlap ratio $r_i = n_{\cap,i}/n_{\min,i}$
 - 10: $\text{Overlap}_p = \text{mean}_i(r_i)$ over all valid positions i **return** Overlap_p
-

Algorithm 5 Masked Mean Reciprocal Rank (MRR) Computation

Require: Teacher probabilities $T \in \mathbb{R}^{N \times V}$, student probabilities $S \in \mathbb{R}^{N \times V}$, mask $M \in \{0, 1\}^N$

Ensure: Masked MRR MRR

- 1: For all positions i where $M[i] = 1$:
 - 2: Find the correct token index: $y_i^* = \operatorname{argmax}_v T[i, v]$
 - 3: Sort $S[i]$ in descending order, get ranked indices R_i
 - 4: Find rank r_i of y_i^* in R_i (i.e., $R_i[r_i] = y_i^*$)
 - 5: Compute reciprocal rank: $rr_i = 1/(r_i + 1)$
 - 6: MRR = $\operatorname{mean}_i(rr_i)$ over all valid positions i **return** MRR
-

E Model Inference with UMIM On-the-fly

We provide two algorithms for integrating UMIM into the inference process of pre-trained language models Section 3.4. Rather than being mutually exclusive or strictly alternative, these algorithms offer flexible integration strategies that can be selected based on deployment requirements or resource constraints. Importantly, both algorithms are compatible with a wide range of autoregressive decoding strategies, such as greedy decoding, top- k sampling, and top- p (nucleus) sampling. Our approach does not impose any restrictions on the choice of decoding algorithm; UMIM operates independently of the underlying generation method and can be seamlessly incorporated into any autoregressive inference pipeline. This design ensures broad applicability, allowing practitioners to benefit from token merging and context compression regardless of their preferred decoding configuration.

Prompt-Only Merging In this approach, UMIM is applied a single time to the input prompt prior to the start of generation. Specifically, we scan the entire tokenized prompt for all eligible n -grams as defined in the merge set $\mathcal{R}$, and each matching span is replaced with its corresponding surrogate embedding, generated via our multi-head pooling mechanism. This transformation produces a compressed sequence of embeddings that serves as the model’s input. The compressed prompt is then fed into the language model, after which generation proceeds in the standard autoregressive fashion without any further merging or intervention by UMIM. Because token merging is performed only once on the initial prompt, this strategy leads to an immediate reduction in both the effective prompt length and the size of the initial key-value (KV) cache required for generation. As a result, prompt-only merging yields significant efficiency gains in terms of memory and computation, all while preserving full compatibility with any standard autoregressive decoding algorithm.

Pseudocode 2: Prompt-Only Merging

```
# Prompt-Only Merging
# x: input text
# R: merge rules
# NAMEM: trained merge module
# LM: pretrained language model

z = Tokenizer(x)
z_m = z.copy()

for n in [4, 3, 2]:
    while True:
        found = False

        for i in range(len(z_m) - n + 1):
            span = tuple(z_m[i:i+n])

            if span in R[n]:
                z_m = (
                    z_m[:i]
                    + [span]
                    + z_m[i+n:]
                )
                found = True
                break
```

```

        if not found:
            break

E = []

for e in z_m:
    if isinstance(e, tuple):
        span_embs = [
            model_embedding(t) for t in e
        ]
        emb = NAMEM(stack(span_embs))
    else:
        emb = model_embedding(e)

    E.append(emb)

E = stack(E)
y = LM.generate(inputs_embeds=E)

return y

```

Dynamic Online Merging During Generation In the second approach, UMIM is applied dynamically throughout autoregressive decoding. After each new token is generated, we examine whether the most recent n -gram (ending with the new token) matches any entry in $\mathcal{R}$. If a match is found, we roll back the last n steps: removing their KV cache entries and replacing them with a single surrogate embedding produced by UMIM. This embedding is re-encoded and reinserted into the cache, allowing the model to continue generation with a compressed sequence. This on-the-fly merging strategy incrementally reduces context length, minimizing memory and computation cost as generation progresses.

Pseudocode 3: Dynamic Online Merging

```

# Dynamic Online Merging
# x: input text
# R: merge rules
# NAMEM: trained merge module
# LM: pretrained language model

z = Tokenizer(x)

# Apply prompt-only merging first
z_m = apply_prompt_merging(z, R)

# Build compressed prompt embeddings
E = []

for e in z_m:
    if isinstance(e, tuple):
        span_embs = [
            model_embedding(t) for t in e
        ]
        emb = NAMEM(stack(span_embs))
    else:
        emb = model_embedding(e)

    E.append(emb)

E = stack(E)

# Prefill LM with compressed prompt
logits, C = LM.forward(
    inputs_embeds=E,
    cache=None
)

# Buffer for recently generated raw tokens

```

```

recent_tokens = []

while not finished:
    y = sample(logits)

    z_m.append(y)
    recent_tokens.append(y)

    merged = False

    for n in [4, 3, 2]:
        if len(recent_tokens) < n:
            continue

        span = tuple(recent_tokens[-n:])

        if span in R[n]:
            z_m = z_m[:-n] + [span]

            # y has not been forwarded yet, so only
            # the previous n-1 tokens have KV entries.
            C = remove_last_n(C, n - 1)

            span_embs = [
                model_embedding(t) for t in span
            ]
            emb = NAMEM(stack(span_embs))

            logits, C = LM.forward(
                inputs_embs=emb,
                cache=C
            )

            recent_tokens = []
            merged = True
            break

    if not merged:
        emb = model_embedding(y)
        logits, C = LM.forward(
            inputs_embs=emb,
            cache=C
        )

output = decode(z_m)
return output

```

F Implementation Details

F.1 Training Data Sets

To train our merge module across various base models—including Llama 3.1 8B, Llama 3.2 1.5B, and GPT2-XL 1.5B—we utilize the WikiText-103 corpus. We preprocess all texts and concatenate them into a fixed-length dataset, where each sequence has a length of 512 tokens. For the DeepScaleR-1.5B-Preview model, we use a custom-generated dataset. Specifically, we leverage questions from two prominent reasoning benchmarks: KbsdJames/Omni-MATH and RUC-AIBOX/STILL-3-Preview-RL-Data. For each question, we construct a prompt consisting of the question text and then perform three independent sampling runs, generating additional tokens such that the combined length of the prompt, question, and generated output reaches 1024 tokens per sample. All samples are concatenated to form a fixed-length dataset, where each sequence is exactly 1024 tokens.

F.2 Parameter Settings and Training Implementation Choices

General Implementation Details All experiments were conducted on 8 NVIDIA GPUs (a mix of RTX A6000, RTX 6000 Ada, and L40S, each with 48GB memory). For distributed training, either PyTorch DistributedDataParallel (DDP) or DeepSpeed ZeRO stage 2 in bf16 precision was used, depending on the model. The optimizer was AdamW (learning rate 8×10^{-4} , weight decay 1×10^{-3}), with learning rate warmup and decay. Each model was trained for 15 epochs. The datasets were preprocessed, saved as .pt or .pkl files, and loaded via a custom TextDataset with DistributedSampler. The teacher model for distillation was frozen, and only the merge module parameters in the student model were updated. Checkpoints were saved after each epoch, with the best model selected by validation loss and early stopping (patience = 3). All metrics—including cross-entropy loss, top-1/top-3/top-10 accuracy, top- p ($p = 0.9$), and MRR—were logged with Weights & Biases (wandb).

Model-Specific Settings

- **Llama 3.1-8B**: DeepSpeed ZeRO stage 2, batch size 3 per GPU, sequence length 512, merge module $d_{\text{model}} = 4096$.
- **Llama 3.2-1.5B**: DDP, batch size 4 per GPU, sequence length 512, merge module $d_{\text{model}} = 2048$.
- **GPT2-XL**: DDP, batch size 4 per GPU, sequence length 512, merge module $d_{\text{model}} = 1600$.
- **DeepScaleR-1.5B-Preview**: DeepSpeed ZeRO stage 2, batch size 3 per GPU, sequence length 1024, merge module $d_{\text{model}} = 1536$.

G Practical Note on Full-Vocabulary Distillation

We include this section as a practical training note rather than a methodological contribution. A natural question in our training design is why we employ *full-vocabulary* distribution distillation, instead of the commonly used Top- k distillation (e.g., Top-256/Top-512). In standard knowledge distillation for language modeling, restricting the teacher distribution to a relatively small Top- k set is often sufficient and sometimes even beneficial. This is because next-token distributions produced by well-trained LMs are typically sharp: most probability mass concentrates on a small subset of tokens, and the long tail carries little additional information for aligning the student model.

However, our setting differs from conventional distillation in a fundamental way. The student in our case is not another language model of similar structure, but a *merge module operating directly in the input embedding space*. We freeze the backbone LM and train the merge module to replace a multi-token span with a single *super embedding*, such that the frozen LM’s next-token predictive distribution under the merged prefix matches that under the original prefix. As a result, the *only* supervision signal available to the merge module is the gradient propagated through the frozen LM from logits back to the input embeddings. There is no intermediate supervision (e.g., attention maps, hidden-state alignment, or layer-wise matching). Consequently, any truncation of the teacher distribution directly truncates the already indirect and fragile optimization signal.

Support mismatch in early training. Top- k distillation implicitly assumes that the student distribution is already reasonably close to the teacher distribution, at least on the teacher’s high-probability support. In typical LM-to-LM distillation, this assumption is often approximately satisfied: both teacher and student operate on the same tokenization and similar computational pathways, so their predictions tend to overlap substantially even early in training.

In contrast, the merge module produces embeddings that are *not* guaranteed to lie on the input manifold induced by the original tokenization. Early in training, replacing multiple tokens with a single learned embedding typically introduces a structured distribution shift, and the resulting next-token distribution can deviate significantly from the teacher. In such a regime, the teacher’s Top- k set may have limited overlap with the student’s high-probability tokens. Truncating the distillation target to the teacher’s Top- k therefore creates an immediate *support mismatch*: many tokens that the student assigns substantial probability to are omitted from the loss, and thus receive no corrective gradient.

Gradient truncation and loss discontinuity. This support mismatch has two concrete optimization consequences that are particularly harmful for embedding-level merging:

(i) Gradient truncation on precisely the error modes that matter. When the student assigns probability mass to tokens outside the teacher’s Top- k , Top- k distillation discards those tokens by construction. This removes a large portion of the gradient signal that would otherwise penalize the student’s incorrect mass allocation. For a merge module whose only learning signal comes from logits, this truncation is critical: it prevents the module from learning how to *pull mass out of wrong tokens* and redistribute it toward the teacher distribution. Empirically, this manifests as stalled learning or unstable oscillations.

(ii) A non-smooth objective induced by a discrete Top- k operator. Top- k selection is a discrete, piecewise-constant operation on the teacher logits. Small changes in the merged embedding can cause abrupt changes in which tokens fall inside the teacher’s Top- k set (or, similarly, which tokens dominate the student distribution), leading to a loss surface that is effectively non-smooth. In standard distillation this effect is often muted because the student stays close to the teacher and the Top- k set changes slowly. In our setting, where the merged embedding can initially be far from the teacher-induced representation, Top- k membership can change frequently, injecting high variance into gradients and further harming convergence.

Why full-vocabulary distillation is necessary here. Full-vocabulary distillation avoids both issues above. By matching the complete teacher distribution, we preserve a continuous training signal over the entire vocabulary support. Even when the merge module initially produces poor predictions, the loss provides gradients that penalize incorrect probability mass wherever it appears (including in the long tail), enabling the merged embedding to move steadily toward a behavior-preserving representation. This is particularly important because the merge module is so

H Additional Experiments and Evaluation

H.1 Text Generation

H.1.1 Text Generation Setup

We describe our experimental setup for text generation with Llama 3 8B:

For qualitative results, we randomly select five prompts from the WikiText-103 test split. For each prompt, we generate either 20 or 30 tokens, depending on the length required to illustrate a complete sequence, using greedy decoding. Additionally, we select one prompt from the WikiText-103 test split and perform top- p sampling with $p = 0.9$ and temperature 0.8. For this prompt, we sample three independent generations.

For language modeling performance, we use WikiText-103, BookCorpus, and OpenWebText. We directly compute the perplexity for both the base model and our merge model on these datasets. The PPL calculation process for the merge module is described in (See algorithm 4).

For quantitative results, we use the WikiText-103 test split and report metrics including Rep-2, Dist-2, Rep-3, Dist-3, Rep-4, Dist-4, ROUGE-2, ROUGE-L, ROUGE-Lsum, BERTScore, chrF, and Mauve. For each sample, the first 100 tokens are used as the prompt, and the next 100 tokens (tokens 101–200) as the reference. The model is conditioned on the prompt to generate 100 tokens, which are then compared against the reference sequence. We evaluate three decoding strategies: greedy decoding, top- p sampling (with $p = 0.9$, temperature = 0.8), and random sampling. For both top- p and random sampling, we perform five independent generation runs for each input and report the average results.

Pseudocode 4: Two-Stage PPL Calculation

```
# Two-stage PPL for dynamic merging
# x: token IDs, length N + 1
# R: merge rules
# NAMEM: merge module
# LM: pretrained language model

z_m, records = merge_with_context(x, R)
```

```

# Stage 1: merged-sequence loss
L1 = 0
inputs = z_m[:-1]
targets = []

for i in range(len(inputs)):
    nxt = z_m[i + 1]

    if isinstance(nxt, tuple):
        targets.append(nxt[0])
    else:
        targets.append(nxt)

E = []

for e in inputs:
    if isinstance(e, tuple):
        emb = NAMEM(e)
    else:
        emb = model_embedding(e)

    E.append(emb)

logits = LM.forward(stack(E))

for t, target in enumerate(targets):
    L1 += -log_prob(logits[t], target)

# Stage 2: skipped-token loss
L2 = 0

for _, rec in records.items():
    ctx = rec["context"]
    cur = rec["current_token"]
    tgt = rec["target_token"]

    E_ctx = []

    for c in ctx:
        if isinstance(c, tuple):
            emb = NAMEM(c)
        else:
            emb = model_embedding(c)

        E_ctx.append(emb)

    E_ctx.append(model_embedding(cur))

    logits = LM.forward(stack(E_ctx))
    L2 += -log_prob(logits[-1], tgt)

# Total loss and perplexity
L_total = L1 + L2
PPL = exp(L_total / (len(x) - 1))

```

H.1.2 Quantitative Results

We present comprehensive qualitative generation results as shown in Tables 8 and 9. Across all decoding strategies, we observe that the merge module can effectively condense the semantic content of multiple tokens into a single surrogate token, with minimal impact on the fluency or coherence of the generated text.

Importantly, the merged tokens are not simply unimportant or redundant. During autoregressive generation, merged tokens retain substantial influence, as evidenced by their significant attention weights in the model’s output. This indicates that the model continues to rely on merged tokens for contextual reasoning and content prediction, rather than disregarding them. Visualization of attention

distributions further supports that merged tokens play an active and meaningful role in the generative process, as indicated in Figure 4.

In addition, embedding similarity analysis shows that the representation of a merged token exhibits near-zero cosine similarity with the embeddings of its original constituent tokens. This demonstrates that the merge module learns to encode a distinct, context-aware representation for each merged span.

H.1.3 Quantitative Results

Quantitative metrics—including n-gram repetition, diversity (Rep- n , Dist- n), and multiple text similarity scores (ROUGE, BERTScore, chrF, Mauve)—confirm that the merge model maintains generation quality comparable to the base model across all decoding settings. Overall, our results demonstrate that the merge module enables effective semantic compression, while preserving the expressiveness, interpretability, and attention dynamics of the underlying language model.

Table 8: Qualitative generation results with greedy decoding. Highlighted texts are merged with UMM, with [] for merging boundaries.

Prompt	Generation	
	Base Model	Merge Module
Traditional Chinese literary criticism emphasized [the life of the] author when interpreting a work	The author’s life was seen as the key to understanding the work.	The author’s life [was seen as] the key [to understanding the work] .
In [2025] , [he moved to the] institute [in an attempt] to study.	He was a very good student, and he was very good at his job.	He [was a student of] [the institute and] [was a member of] the institute’s football team.
In the spring, [his youngest son] , Ian, was born.	He was a beautiful baby boy, and he was the light of his father’s life.	He [was a beautiful baby] , [with a head of thick] , dark [hair and] a round, chubby face.
The first public library in Bintulu [was built in] 1971.	It was located at the Bintulu District Office. The library was later moved to the Bintulu Town Council building in 1973.	It [was located at] the Bintulu District Office. The library was [later moved to the] Bintulu Town Council building [in 1973] .
Everyone considered the reviewers [an improvement on the] conference.	The reviewers were very helpful and the reviews were very thorough. The reviewers were very helpful and the reviews were very thorough.	The reviewers were very professional [and the conference] was very well organized. The conference [was a great success] and we are looking forward to next year’s conference.

H.2 Downstream Tasks

We strictly follow the framework of LLM Evaluation test QA. We combine prompts, questions, and answers into a sequence. Then we calculate the loss of each overall context. We select the option with the lowest loss as the correct answer and compare it with the correct answer.

For the CNN/DailyMail task, we conducted experiments using the LLaMA-8B model. Consistent with previous setups, token merging was guided by merge rules identified from the WikiText training split. All outputs were generated using greedy decoding with a fixed length of 80 tokens. To ensure comparability with baselines, token merging was applied exclusively to the input portion of each example. For the Select Context baseline, token-level reduction of the prompt was performed. Regarding LLMingual, we employed its default token reduction method, while for LLMingual2, we utilized its small model variant for prompt reduction. For the Sink Attention baseline, we configured 4 sink tokens and a sliding window equal to 16% of the total length (prompt length plus generated tokens). Due to the inherent difficulty in precisely controlling the token reduction with

Table 9: Qualitative generation results with Top-p decoding. Highlighted texts are merged with UMIM, with [] for merging boundaries.

Prompt: Tell me something about Lakers.			
	Sample 1	Sample2	Sample3
Base	Why did they change their name from Minneapolis Lakers to Los Angeles Lakers? What are their nicknames? What are their logos? Who are their rivals? Who are their greatest players? What are their colors? What is their motto?	What do you think of Kobe? Lakers are the best team in the league, I believe. I think they will win the championship again. Kobe is the best player in the league, and I think he will win the MVP this year.	I’ve heard that they are the best team in the NBA, but I’m not sure about that. Who are the best players on the team? What are their chances to win the NBA championship this season? I want to see them in action.
Merge	Tell me something about Lakers. I never [been to a] game. I want [to know everything] about [the team and the] players [and the history of] the team. I’m like [a kid in] [a candy store] . [I just want to] know everything.	What [do you think of] their chances [to make the playoffs] this year? What [do you think of] [the Clippers] ? Do you think they’ll be [able to win the] championship this year? What [do you think of] the Lakers’ chances?	What is [the most exciting thing] about Lakers? Why do you love them? What makes [them special] ? The Lakers [are a very] special team. They have had [some of the greatest] players [to ever play the] [game of basketball] .

Table 10: Quantitative results of N-gram repetition and diversity metrics

Method	Rep-2	Dist-2	Rep-3	Dist-3	Rep-4	Dist-4
Top- <i>p</i> (Base model)	11.76	39.58	10.93	70.14	6.78	85.59
Top- <i>p</i> (Merge Model)	11.28	43.49	9.08	75.59	4.55	89.92
Greedy (Base model)	14.03	40.99	15.32	60.38	13.97	69.75
Greedy (Merge Model)	13.34	39.04	14.44	58.96	12.74	69.30
Sampling (Base model)	11.06	50.72	7.13	83.10	2.84	94.68
Sampling (Merge Model)	10.77	33.96	12.02	59.34	9.76	74.25

Table 11: Quantitative results of text similarity

Method	Rouge-2	Rouge-L	Rouge-Lsum	BERTScore	chrF	Mauve
Top- <i>p</i> (Base model)	5.49	17.85	19.66	83.75	28.53	34.87
Top- <i>p</i> (Merge Model)	4.87	17.17	19.14	83.57	28.39	33.90
Greedy (Base model)	5.82	18.89	20.03	83.35	26.99	64.28
Greedy (Merge Model)	5.38	18.44	19.47	82.98	26.08	67.21
Sampling (Base mode)	4.12	16.23	18.24	83.25	28.28	36.81
Sampling (Merge Model)	5.45	18.09	19.58	83.44	27.67	27.41

LLMLingua and Select Context, their reported token reductions are approximate. In contrast, our approach ensures precise control over the exact number of merged tokens. The experimental results clearly demonstrate that our token merging method consistently outperforms all baselines, even in a domain entirely unseen during training. This confirms not only the strong merging capability of our module but also highlights its generalization and robustness across downstream tasks beyond the original training domain. We conducted evaluations on the AIME2024 and AMC mathematics benchmarks using the DeepScaleR model. Given that reasoning tasks typically require generating a large number of tokens, computational resource limitations prevented extensive training involving such lengthy outputs. Therefore, token merging was applied exclusively to the input prompts. We set the maximum generation length to 15,000 tokens and assessed performance using the pass@k(k=16) accuracy metric. Experimental results indicate that our merge module consistently achieves strong performance on these challenging reasoning tasks. Notably, our method outperforms both Select

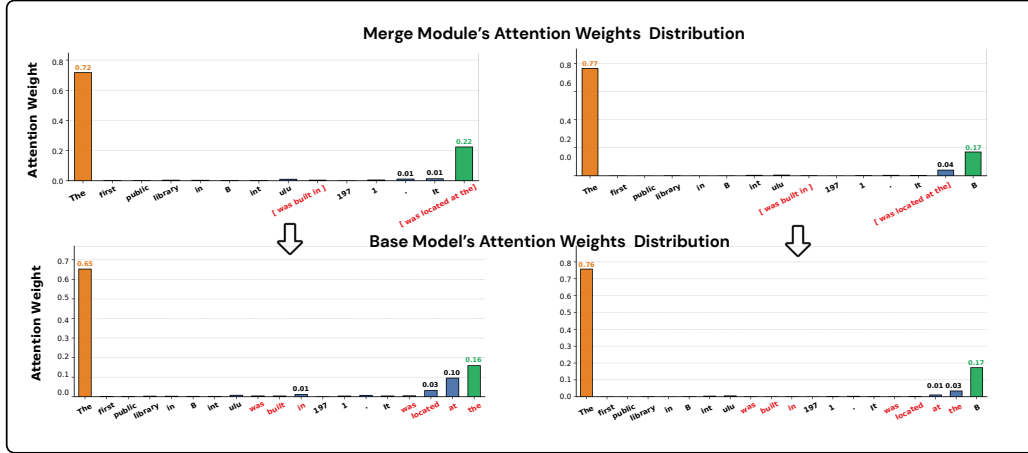


Figure 4: Attention distribution after applying the merge module at inference compared with the original token-based inference.

Context and LLMingua baselines, further underscoring its effectiveness and generalizability in reasoning-intensive applications.

H.3 Supplementary Evaluation on Specialized Domains: Domain-Adapted UMIM

We first emphasize that the primary goal of the experiments in the main paper is to validate the **generality** of the proposed merge module in general-domain settings. In these evaluations, both the merge set and the UMIM module are constructed and trained solely on WikiText, and are then *directly applied to downstream tasks without any task-specific fine-tuning*. The stable performance across diverse downstream tasks demonstrates that UMIM is a general-purpose merging method rather than a heuristic tuned to a single benchmark.

We stress that this domain-adapted experiment should not be interpreted as evidence against the generality of UMIM. The main paper already evaluates the WikiText-trained UMIM under direct transfer, without task-specific fine-tuning, across diverse general-domain tasks. In contrast, code generation represents a substantially different domain, with specialized syntax, symbolic structure, and token statistics. It is therefore not a meaningful requirement that merge rules mined from general-domain text must directly provide high compression on code. Such a requirement would conflate general-domain transfer with cross-domain adaptation.

Our goal in this supplementary study is instead to test whether the UMIM formulation remains effective when the merge candidates are constructed from the target domain itself. For this purpose, we evaluate a domain-adapted setting in which both the merge rules and the merge module are constructed using in-domain code data. This setting tests whether the proposed merging formulation remains effective once the merge candidates reflect the statistical regularities of the target domain.

For this purpose, we construct a code-specific merge set from the DeepMind CodeContests corpus and train a code-specific UMIM module. This experiment is not intended to replace the general-domain evaluation, but to test the adaptability of UMIM under domain-matched merge statistics. Due to time and computational constraints, the amount of code training data is limited; therefore, we view this experiment as a supplementary validation rather than a fully optimized code-compression system.

We evaluate the code-adapted UMIM on HumanEval using Llama-3.1-8B under a fixed 25% merge ratio. The results are summarized in Table 12. Despite the limited training data, UMIM retains approximately 81% of the original pass@1 performance (62.2% $\rightarrow$ 50.6%). In contrast, under the same compression ratio, LLMingua-2 and Selective-Context almost completely fail.

These results indicate that, when the merge set and merge module are adapted using in-domain data, UMIM can still achieve meaningful compression while preserving a substantial portion of the model’s

Method	pass@1 (%)	Passed	Failed
Baseline (Llama-3.1-8B)	62.2	102	62
UMIM (25% merged, code-trained)	50.6	83	81
LLMLingua-2 (25% merged)	1.2	2	162
Selective-Context (25% merged)	0.0	0	164

Table 12: HumanEval results (164 tasks) on Llama-3.1-8B under an identical 25% merge ratio. UMIM retains approximately 81% of the original pass@1 performance, while LLMLingua-2 and Selective-Context nearly collapse under the same compression setting.

capability, even in structurally sensitive code-generation tasks. Together with the main experiments, this suggests that UMIM is not tied to a particular corpus, but provides a flexible merging formulation that can be applied either directly in general-domain settings or adapted to specialized domains when their token statistics differ substantially.

I Additional Details on Task-Specific Adaptation

In the main paper, we briefly describe task-specific adaptation as a lightweight way to improve the accuracy–compression trade-off of UMIM. Here we provide a more detailed explanation of the adaptation protocol and clarify the role of each setting. The goal of this appendix section is not to introduce a separate method, but to make explicit how the adaptation experiments are conducted and why the observed trend is not specific to a single benchmark.

I.1 Detailed Adaptation Protocol on HellaSwag

We first detail the adaptation pipeline used for HellaSwag. Throughout all settings, the backbone LLM is kept completely frozen. No parameter of the language model is updated during task-specific adaptation. All adaptation is restricted to two lightweight components: (1) the construction of merge rules from the target-task training split, and (2) the training or fine-tuning of the merge module.

Task-specific merge rules. For HellaSwag, we construct merge rules using only the HellaSwag training split. Validation and test examples are not used for rule construction or adaptation. To make the comparison across compression levels more controlled, we use only bigram merge rules in this experiment. The merge rules are obtained by simple frequency counting over adjacent token pairs, followed by deduplication and longest-match application at inference time. This rule-construction step is lightweight and does not involve training the backbone LLM.

We use frequency thresholds $\{2000, 200, 50, 5, 1\}$ to obtain different merge rates. These thresholds approximately correspond to merge rates of 10%, 20%, 30%, 40%, and 50%, respectively. Lower thresholds allow more bigrams to be merged and therefore produce more aggressive compression. In particular, threshold = 1 is the most aggressive setting, where nearly all eligible adjacent bigrams can be selected as merge candidates.

Adaptation settings. We compare several settings to separate the effects of rule reconstruction, supervised fine-tuning, and preference-based adaptation.

Task-trained denotes training a merge module from scratch on HellaSwag using HellaSwag-specific merge rules. In this setting, we reinitialize the merge module and train it on the HellaSwag training split, following the same distillation-style training procedure used for the original WikiText-trained module, but replacing WikiText with task-specific data.

WT + Rules keeps the WikiText-pretrained merge module fixed, but replaces the WikiText-derived merge rules with HellaSwag-specific merge rules. This setting tests whether the pretrained merge module can directly operate under a new task-specific rule set without any additional gradient-based training.

WT + FT further fine-tunes the WikiText-pretrained merge module after replacing the merge rules. Only the merge module is updated, while the backbone LLM remains frozen. In practice, this adaptation is fast and typically becomes effective within 2–3 epochs.

WT + DPO applies Direct Preference Optimization directly to the WikiText-pretrained merge module without supervised fine-tuning first. This setting is included to test whether preference optimization alone is sufficient for adaptation. Empirically, we find that direct DPO can be unstable, especially under heavier compression, suggesting that DPO benefits from a better adapted initialization.

Finally, Task-Adapted denotes the full adaptation route: WT + Rules + FT + DPO. That is, we first replace the merge rules with HellaSwag-specific rules, then fine-tune the merge module, and finally apply DPO to further improve the accuracy–compression trade-off.

Interpretation. The progression across these settings shows that task-specific adaptation improves the compression mechanism without fine-tuning the language model itself. Rule reconstruction improves merge coverage by matching the candidate spans to the target task. Fine-tuning then adjusts the merge module so that the surrogate embeddings better preserve the frozen LLM’s predictive behavior under the new rule set. DPO provides an additional preference-based refinement when applied after supervised fine-tuning.

We compare with external prompt-compression baselines at comparable merge rates. We note that methods such as LLMingua-2 are also based on trained compression models: they learn a token-level compression policy to decide which prompt tokens should be retained before passing the shortened prompt to the backbone LLM. Similarly, UMIM does not update the backbone LLM. Instead, it adapts a separate lightweight merge module and reconstructs merge rules from the task training split. Therefore, the comparison should be understood as comparing different trained compression mechanisms under a frozen backbone LLM, rather than comparing a fine-tuned language model against an unfine-tuned one.

I.2 Additional ARC Results

We further include ARC results to verify that the adaptation trend observed on HellaSwag is not an isolated outcome of a single benchmark. The purpose of this experiment is not to introduce a new task setting, but to test whether the same pattern holds beyond HellaSwag.

Figure 5 presents the accuracy–compression trade-off on ARC-Easy and ARC-Challenge. The WikiText-pretrained merge module (WT) already shows non-trivial transfer, maintaining competitive accuracy under moderate compression. This indicates that the pretrained merge module captures reusable merging behavior rather than relying on a benchmark-specific heuristic.

More importantly, the transition from WT to Task-Adapted consistently improves the accuracy–compression trade-off. The upward shift shown in Figure 5 mirrors the trend observed on HellaSwag: reconstructing task-specific merge rules and lightly adapting the merge module improves performance while preserving compression. This consistency suggests that the adaptation pipeline is not merely a lucky fit to HellaSwag, but reflects a stable property of the proposed method.

Overall, the ARC results support the same conclusion as the HellaSwag analysis: UMIM can be directly transferred with a WikiText-trained merge module, and its performance can be further improved through lightweight task-specific adaptation without updating the backbone LLM.

I.3 Efficiency Analysis

I.4 Empirical Efficiency Evaluation and Throughput Analysis

While GPUs provide massive parallel compute capacity, the primary bottleneck in autoregressive decoding is not arithmetic throughput but memory I/O. Due to the sequential nature of Transformer decoding, each step generates only a single new token, resulting in relatively small per-step computation. However, each newly generated token must attend to the entire accumulated KV cache, requiring repeated reads of large memory blocks. As a result, KV cache access dominates runtime and memory usage, making the effective sequence length the key limiting factor for inference efficiency.

Evaluation protocol. To directly address the concern that token-level compression does not necessarily translate into faster inference, we conduct explicit throughput measurements inspired by the evaluation protocol of Zip2Zip. We report end-to-end throughput in tokens per second (tokens/sec), measured separately for the *prefill* stage and the *decoding* stage. All experiments are conducted

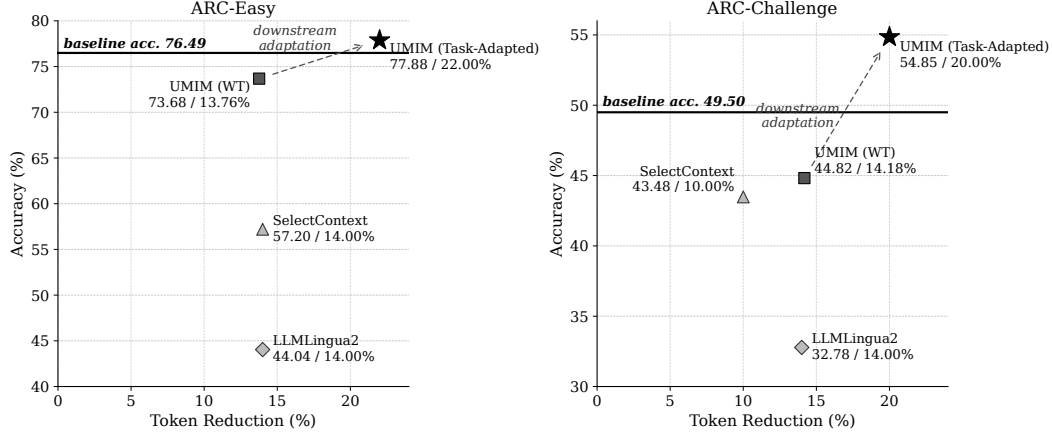


Figure 5: **Accuracy-compression trade-off on ARC-Easy and ARC-Challenge.** Each point shows a method’s accuracy (%) vs. token reduction (%). The dashed horizontal line indicates the uncompressed baseline accuracy. The arrow marks the effect of downstream adaptation from WT to Task-Adapted.

on Llama-3.1-8B under a fixed 50% merge ratio, while varying the prompt context length. Unless otherwise specified, throughput is measured with respect to the original, uncompressed token count. For the merge model, this therefore measures how many original input tokens are covered per second after compression, rather than the number of effective tokens fed into the backbone LM. Therefore, for the merge model, the reported throughput reflects the rate of processing the original prompt before compression, rather than the throughput over the shortened embedding sequence after merging.

Results. Table 13 summarizes the measured throughput. We observe substantial speedups during the prefill stage, ranging from $1.5\times$ to $3.7\times$, with gains increasing monotonically as the context length grows. This shows that, under the original-token throughput metric, reducing the effective input length allows the model to cover more original prompt tokens per second.

During autoregressive decoding, we observe consistent but modest throughput improvements (0.9%–2.5%) across all settings. While smaller in magnitude, these gains are systematic and strictly positive.

Interpretation. The observed behavior aligns closely with our FLOPs and memory analysis. Prefill merging yields an immediate one-time reduction in sequence length, producing large and readily observable speedups. In contrast, decoding is fundamentally bandwidth-bound and benefits from merging only through incremental reductions in KV cache size. Under our current prototype implementation, decoding involves Python-level control logic for token merging, embedding pooling, and KV cache rollback, which introduces scheduling overhead and frequent small kernel launches. Despite this non-fused control overhead, decoding throughput still improves, indicating that the savings from reduced attention computation already offset the additional control cost.

Importantly, the modest decode-side speedup should not be interpreted as a limitation of the merging formulation itself. Rather, it reflects an implementation-level bottleneck: merging and cache management are not yet fused into optimized GPU kernels. Once these operations are integrated into end-to-end CUDA or Triton kernels, the realized decoding speedup is expected to more closely match the theoretical gains predicted by KV cache reduction.

Upper-bound potential. Notably, surrogate embeddings depend solely on static token identities and can be precomputed offline and stored in a lookup table. This design eliminates runtime embedding computation and further reduces control overhead. Such optimizations remove implementation inefficiencies without altering the underlying algorithm, suggesting that the current measurements represent a conservative lower bound on achievable acceleration.

Setting	Prefill Throughput (tokens/sec)			Decode Throughput (tokens/sec)		
	Base	Merge	Rel. (%)	Base	Merge	Rel. (%)
256 + 256	10531.8	15410.8	+46.3	64.7	65.6	+1.3
512 + 256	14210.6	30978.5	+118.0	65.3	65.9	+0.9
1024 + 256	16138.3	50516.0	+213.0	65.1	66.4	+1.9
2048 + 256	16592.5	62124.7	+274.4	64.8	66.4	+2.5

Table 13: Throughput comparison (original tokens/sec) for Llama-3.1-8B under a fixed 50% merge ratio. Prefill throughput shows large gains that scale with context length, while decoding throughput exhibits consistent positive improvements despite Python-level control overhead.

Summary. Overall, these results provide direct empirical evidence that reducing the effective sequence length via token merging leads to real inference speedups. Prompt merging delivers large immediate gains, while generation-time merging yields consistent positive improvements even under a non-optimized implementation. Together, these findings validate that UMIM improves inference efficiency in practice, and that its performance is primarily limited by current engineering choices rather than by the method itself.

J Ablation Study

J.1 Different Thresholds of N-grams

In summary, our results clearly show that as the proportion of merged tokens increases, the training metrics consistently decrease. This relationship is direct and inevitable: a lower merge threshold τ leads to more frequent merging, which in turn results in a larger portion of the sequence being combined into n-grams and a corresponding decline in model performance. This trend is observed across both Llama-3.2-1.5B and GPT2-XL (See Tables 14 and 15). These findings highlight an inherent trade-off: while merging more tokens can improve efficiency by reducing sequence length, it unavoidably comes at the cost of lower training quality. Therefore, selecting an appropriate merge threshold is essential to balance efficiency and performance in practice.

Table 14: Different Threshold of N-grams -Llama 3.2 1.5B

τ	Top1	Top3	Top10	Top p	MRR
500	0.8437	0.8356	0.8433	0.9504	0.9078
50	0.7883	0.7840	0.7943	0.9320	0.8691
5	0.7410	0.7414	0.7528	0.9155	0.8297

Table 15: Different Threshold of N-grams GPT2 XL

τ	Top1	Top3	Top10	Top p	MRR
500	0.8414	0.8456	0.8579	0.9548	0.9048
50	0.7588	0.7707	0.7893	0.9297	0.8447
5	0.6955	0.7128	0.7359	0.9065	0.7928

J.2 Different Merge Module Sizes

To assess the impact of model capacity, we conduct an ablation study on the size of the merge module. Specifically, we retrain merge modules of different depths on WikiText-103, fixing τ at 5. As shown in Table 17, increasing the number of attention layers from one to three results in only marginal performance improvements across all evaluation metrics. These results indicate that our merge module design is highly parameter-efficient and already achieves strong performance in this scenario.

Table 16: Different Merge Module Size Based on Llama-3.2-1.5B

Merge Module Size	τ	Top-1	Top-3	Top-10	Top- p	MRR
One Attention Layer	5	74.10	74.14	75.28	91.55	83.30
Three Attention Layers	5	74.27	74.15	75.34	91.58	83.41

J.3 Different Decoding algorithms Generation Results

We employ various decoding strategies, including greedy decoding, top-p sampling, and random sampling, to generate context. The corresponding results are presented in Section H. Overall, our approach does not lead to substantial losses in generation quality.

J.4 Different Merge Rules

We also propose an alternative approach for constructing merge rules by leveraging a language model (Llama-3.2-1.5B) to compute the generation probability of each token within input text chunks. Specifically, we automatically extract all consecutive spans of tokens whose probabilities exceed a predefined threshold (set to 0.5) and are at least two tokens in length. These high-confidence token sequences are collected as merge rules, representing common and reliable language patterns as identified by the model. To ensure compatibility with our decoding algorithm, we discard any candidate chunk longer than four tokens (i.e., restrict merges to at most four-grams). As a result, the total number of TR is relatively modest. According to training alignment metrics, the merge rules constructed using the method described in CD-LM also yield strong training results. While our method is practical and effective, it is not the only possible strategy—other approaches for constructing merge rules may also be considered.

Table 17: CD LM Merge Rules

Model	TR (%)	Top-1	Top-3	Top-10	Top- p	MRR
Llama-3.2-1.5B	10.3%	91.07	91.04	91.52	97.17	94.96

J.5 Positional Encoding under Token-Span Merging

We do not introduce any special positional encoding mechanism for merged tokens. In common autoregressive LLM implementations, if `position_ids` are not explicitly provided, they are assigned according to the current effective sequence length or KV cache length. That is, when the model has already cached K previous token states, the next input tokens are assigned positions

$$K, K + 1, \dots, K + \ell - 1,$$

where ℓ is the length of the new input. Thus, position ids are tied to the current effective prefix length rather than to the original token indices in the uncompressed raw sequence.

Under UMIM, merged spans occupy a single effective token position. For example, consider an original prefix

$$[x_1, x_2, x_3, x_4],$$

whose standard prefill positions are

$$[0, 1, 2, 3].$$

If x_2 and x_3 are merged into a surrogate token x_{23} , the compressed prefix becomes

$$[x_1, x_{23}, x_4],$$

and the positions assigned by the standard implementation become

$$[0, 1, 2].$$

In other words, the surrogate token x_{23} is treated as one effective input token in the compressed sequence. We do not manually preserve the original positions of x_2 or x_3 .

This design is consistent with our training objective. During distillation, the merge module is trained under the same compressed-position convention used at inference time. For an original sequence

$$[x_1, x_2, x_3, x_4],$$

the compressed sequence

$$[x_1, x_{23}, x_4]$$

is fed to the frozen language model with positions assigned according to its effective length. The surrogate token x_{23} is optimized so that the next-token predictive distribution produced after the compressed prefix matches the distribution produced by the original token-by-token computation after the corresponding raw prefix. Thus, x_{23} is not merely asked to represent the local embedding of x_2 or x_3 ; it is trained to preserve the predictive state induced by the merged span.

The same principle applies during autoregressive decoding. When a newly generated token completes a mergeable span, the KV cache entries corresponding to that span are replaced by a single surrogate state. Subsequent position ids are then assigned according to the updated cache length, following the default cache-length-based rule of the underlying LLM implementation. Therefore, UMIM does not require position re-indexing, position interpolation, or any custom positional encoding scheme. Instead, the merge module learns surrogate embeddings that are compatible with the standard positional behavior of the frozen backbone model.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction summarize the proposed input-level merging framework, the frozen-backbone setting, and the main empirical findings. The claims are supported by the method description and experiments in Sections 3–4

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: We include a separate Limitations section discussing the scope of the current study and directions for broader evaluation and adaptive merging.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not present formal theoretical results or proofs; the contribution is methodological and empirical.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper describes the model backbones, training corpora, merge-rule construction, evaluation metrics, downstream tasks, and baselines needed to reproduce the main experimental results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: Code and trained merge-module checkpoints are not released at submission time. We provide experimental details and use publicly available datasets where possible.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The experimental setup specifies the backbones, training data, merge-rule thresholds, evaluation datasets, metrics, and baseline methods. Additional implementation details are provided in the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: We do not report error bars or statistical significance tests, as the main experiments are conducted under fixed model and greedy evaluation settings.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper reports efficiency analysis and provides the compute setting used for evaluating runtime and memory behavior. Additional resource details are included in the experimental setup or appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research uses existing language models and text datasets for compression and evaluation, and we are not aware of deviations from the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The work may reduce inference cost and improve accessibility of language-model deployment. As a general compression method for language models, it could also inherit risks associated with the underlying models.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release a high-risk generative model or new scraped dataset. The method is a compression module applied to existing models.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper cites the existing models, datasets, and baselines used in the experiments. We follow the corresponding access terms and report public sources where applicable.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not introduce a new dataset or benchmark asset. If code or checkpoints are released later, they will be documented with appropriate usage instructions.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or human-subject studies.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or human-subject studies, so IRB approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are the subject of compression and evaluation, but no LLM is used as an important, original, or non-standard component for developing the core method.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.